

AN AUTOMATED MULTIPLE-CHOICE QUESTION GENERATION USING NATURAL LANGUAGE PROCESSING TECHNIQUES

Chidinma A. Nwafor¹ and Ikechukwu E. Onyenwe²

¹Department of Computer Science, Nnamdi Azikiwe University Awka, Anambra State, Nigeria

¹nwaforchidinmaa@gmail.com and ²ie.onyenwe@unizik.edu.ng

ABSTRACT

Automatic multiple-choice question generation (MCQG) is a useful yet challenging task in Natural Language Processing (NLP). It is the task of automatic generation of correct and relevant questions from textual data. Despite its usefulness, manually creating sizeable, meaningful and relevant questions is a time-consuming and challenging task for teachers. In this paper, we present an NLP-based system for automatic MCQG for Computer-Based Testing Examination (CBTE). We used NLP technique to extract keywords that are important words in a given lesson material. To validate that the system is not perverse, five lesson materials were used to check the effectiveness and efficiency of the system. The manually extracted keywords by the teacher were compared to the auto-generated keywords and the result shows that the system was capable of extracting keywords from lesson materials in setting examinable questions. This outcome is presented in a user-friendly interface for easy accessibility.

KEYWORDS

Natural Language Processing, NLP, Computer-Based Test Examination, CBTE, Automatic Question Generation, AOG, Document, Tokenization, Multiple-Choice Question, MCQ

1. INTRODUCTION

According to [1], automatic question generation (AQG) is the task of automatically generating syntactically sound, semantically correct and relevant questions from various input formats such as text, a structured database or a knowledge base. AQG can be naturally applied in many domains such as Massive Open Online Course (MOOC), setting objective/subjective questions, automated help systems, search engines, Chabot systems (e.g. for customer interaction), and healthcare for analyzing mental health [2]. Despite its usefulness, manually creating meaningful and relevant questions is a time-consuming and challenging task. For example, while evaluating students on reading comprehension, it is tedious for a teacher to manually create questions, find answers to those questions, and thereafter evaluates answers.

In the field of education, questioning is widely acknowledged as an effective instructional strategy to evaluation or assessment of students or the learner at the end of a lesson. Questioning as an instructional tool can be traced back to the fourth century, when Socrates used questions and answers to challenge assumptions, expose contradictions, and lead to new knowledge and wisdom. Since Socrates, questioning was employed to encourage students to use higher-order thinking processes and it is the dominant mode of teacher-student interaction accounted for almost 80% of the total interactions [3]. There are three major types of assessment for evaluating students' learning capabilities – Formative assessment, Interim assessment and Summative assessment. Generating questions that covers most areas of a lesson note would help in Formative and Interim assessment because the Formative assessment helps to improve

students' understanding and performance while the Interim assessment helps teachers identify gaps in student understanding and instruction. Multiple-choice Question (MCQ)s that widely cover all the areas of a lesson note is an important effort to test students on the above assessments and the students' overall cognitive levels held at the end of a lesson. Today, MCQs in the area of Computer-Based Testing Examination (CBTE) is the most commonly used tool for assessing the knowledge capabilities of learners in the field of learning.

Properly constructed MCQs can assess higher cognitive processing of Bloom's taxonomy such as interpretation, synthesis and application of knowledge, instead of just testing recall of isolated facts. Most institutions and Centre of learning have embraced MCQ testing as the foundation of their testing systems, thereby saving cost especially where large numbers of candidates are involved—and it is a format that can provide precision where other measurement options may be lacking such as observing performance.

Effective style of questioning as described by [4] is always an issue to help students attend to the desired learning outcome. Furthermore, to make it effective, balancing between lower and higher level question is a must [4]. Time-consuming factor in setting MCQs hinders achieving the above assessments and good learning outcomes due to MCQs generated tend to cover limited areas of a lesson note from which the MCQs are taken from. Therefore, it is necessary for an automatic system that will bootstrap the existing method of setting MCQs.

Natural Language Processing (NLP) is a branch of artificial intelligence that deals with the interaction between computers and humans using the natural language. The ultimate objective of NLP is to read, decipher, understand, and make sense of the written or textual data in natural languages of human in a manner that is valuable. In this paper, we focus on the setting of MCQs for CBTE through NLP to improve the method of setting MCQs and modification, and for creating a viable question bank for subsequent use by the academicians for their learners. This will ensure an MCQ that includes appropriate questions and options based on the learning objectives and importance of the topics discussed in a lesson material. We used NLP methods, Term Frequency-Inverse Document Frequency (TF-IDF) and N-grams, to extract the most significant words present in a lesson material and the selection does not depend on any vocabulary; these words are keywords that capture the main topics discussed in a lesson material, they also serve as an indication of document relevance for users in an information retrieval (IR) environment.

2. RELATED LITERATURE

[5] developed ArikIturri, an AQG for Basque language test questions. The information source for this question generator consists of linguistically analysed real corpora, represented in XML mark-up language. [6] researched on using NLP on narrative texts to find discourse connectives for AQG. Discourse connectives are words or phrases that indicate relationships between two logical sentences or phrases and suggest the presence of mutually related extended verbal expression. The questions were generated by first extracting the text from the materials supplied by the user using text processing concept of NLP. [7] developed a system which can generate various logical questions from the given text input. The system uses 3-step strategy, viz; Select the best potential set of sentences from the given text input from which we could generate the questions, find the subject and context of the sentence to find its core agenda (Gap Selection) and analyze the best form of question that can be generated from that sentence (Question Formation). [8] proposed a method for automatically generating distractors for multiple-choice English vocabulary questions. The proposed method introduces new sources for collecting distractor candidates and utilises semantic similarity and collocation information when ranking the collected candidates. [9] looked at the development of an automatic factual open cloze question generation system which can generate fill-in-the-blank questions without alternatives. The system first extracts a set of informative sentences from the given input corpus based on Part-of-Speech tagging based rules.

Then, generate questions by omitting the answer-keys which are selected by identifying domain specific words in the sentences. The system also suggests answer hints for the examinees to reduce the number of possible answers that make assessment easier.

3. EXPERIMENTAL METHODOLOGY

The paper is based on the generation of MCQG through Natural Language Processing Techniques.

This is used to process the lesson materials/documents fed by the teacher into a multi-choice questions alongside the answers to each questions. The NLP processes are applied using TF-IDF and N-gram. This system only applies to the text of the document or lesson material in extracting keywords presented by the teacher. The document is converted into text file, loaded into the system memory and stored on the system. The text is split into sentences. The split sentences are tokenized, from which the corpus are built as TF-IDF and N-gram mode. From Figure 1, the raw data is the original text file from the document (lesson material). Noise removal is any piece of text which is not relevant to the context of the data such as stop words. Word Normalization converts high dimensional features or N different features to the low dimensional space or 1 feature, it includes tokenization and stemming. The cleaned text is the end result ready for further experimentation and such is used to analyse and auto-generate multi-choice questions (MCQs). Furthermore, we develop an application interface for easy access and presentation of the MCQs to the teachers for acceptance or rejection.

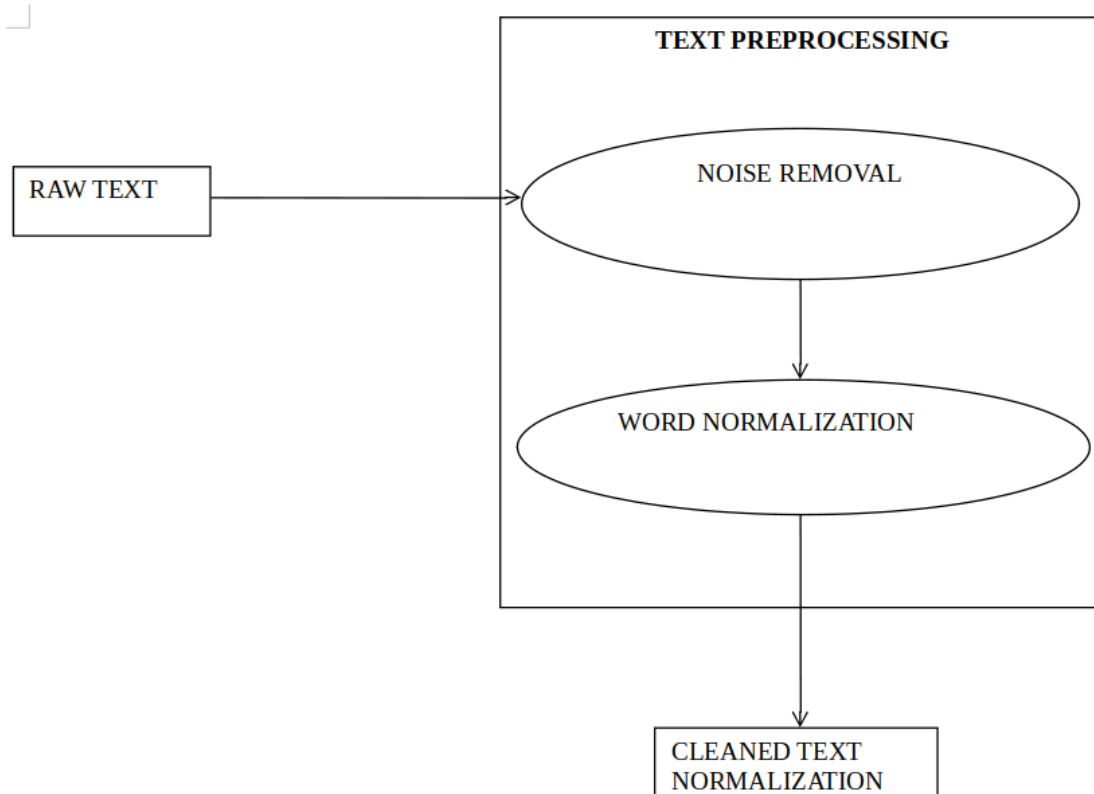


Figure 1. Text Preprocessing of Input Data.

3.1. Dataset and Preparation

In order to compute the performance evaluation of this paper to help check that it is not perverse, keywords extracted by the NLP techniques for accuracy computation was validated. First, five different lecture materials of varying sizes of sentences was selected (see Table 1).

Table1. Statistics of the Textual Data Used

Lesson Materials	Number of Sentences	Number of Words	Number of Keywords by Teachers
Computer Science	39	956	206
Geology	33	903	86
Philosophy	46	820	100
Religion	61	1474	187
History	64	1472	210

In order to evaluate the accuracy of the keywords extraction methods on these lesson materials, it was necessary to collect manually marked keywords to create a gold standard for the evaluation. The usual procedure of creating a gold standard of keywords is having one professional indexer who extracts the keywords based on his expertise and training, but also based on a set of guidelines defined by the interested parties. However, for this experiment, it was felt that the teachers (T) are appropriate for setting a gold standard. They know what words are most relevant in their lecture materials. Each lecture materials were annotated by a T to set a gold standard to which each set of auto-generated keywords could be compared. The statistics of the data used are stated in Table 1.

Table 2. Descriptive statistics of word counts.

Lesson Materials	Minimum Number of Sentences	Maximum Number of Words	Mean number of Sentence
Computer Science	6.0	59.0	24.5
Geology	10.0	43.0	25.8
Philosophy	1.0	39.0	17.8
Religion	1.0	59.0	24.2
History	1.0	79.0	23.0

Table 2 explains the columns “number of Sentences” and “number of words” in Table 1. For instance

Computer Science in Table 3.2, the least sentence has 6-word length of size while maximum sentence has 59-word length of size. Summation of all the sizes of the sentences divided by the

total number of sentences gives the mean size of sentences in Computer Science. This helps gain insight on how words are formed in the sentences of a lecture material. For example, sentences of “Minumum number of Sentence” < 5 are removed on the preprocessing stage.

3.2. Experimental Tools

The following NLP techniques are used for automated generation, manipulation and analysis of data from Figure 1. Data here is the lesson materials from teachers henceforth referred to as documents. In this paper, we represent each sentence in a lesson material as a document. For the auto-generated MCQ’s application interface, we used Django. It is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. It also takes care of much of the hassle of Web development and focuses on writing the API without needing to reinvent the wheel

3.2.1. Term Frequency-Inverse Document Frequency

Term Frequency-Inverse Document Frequency (TF-IDF) is used for stop-words filtering in various subject fields including text summarization and classification. The weight is composed of two terms: Term frequency (TF) and Inverse Document Frequency (IDF).

- TF: Term Frequency measures how frequently a term occurs in a document. It is possible that a term would appear much more times in long documents than shorter ones because every document is different in length. Thus, it is stated below as a way of normalization:

$$TF_t = \frac{\text{Number of times term } t \text{ appears in a document}}{\text{Total number of terms in the document}}$$

- IDF: Inverse Document Frequency measures how important a term is. While computing TF, all terms are considered equally important. However, it is known that certain terms like ‘is’, ‘of’, and ‘that’, may appear a lot of times but have little importance. Thus, it is computed as follows:

$$IDF_t = \text{Log}_e \left(\frac{\text{Total number of documents}}{\text{Number of documents with term } t \text{ in it}} \right)$$

- The TF-IDF_{t,d} weighting scheme assigns to term t a weight in document d given by

$$TF-IDF_{t,d} = TF_{t,d} \times IDF_t$$

3.2.2. N-gram

This is a contiguous sequence of n items from a given sequence of text or speech. The items can be phonemes, syllables, letters, words or base pairs according to the application. The n-grams typically are collected from a text or speech corpus. An n-gram of size 1 is referred to as a ‘unigram’; size 2 is a ‘bigram’; size 3 is a ‘trigram’. Larger sizes are sometimes referred to, by the value of n, for example, ‘4-gram’, ‘5-gram’, and so on.

4. RESULT DISCUSSION

This discusses the performance of the NLP techniques for MCQs generation applied to the dataset of different lecture materials contents (see Table 1) and compared with the gold standard created by T. Table 3 shows the result of application of the above techniques on documents stated in Table 1 after going through preprocessing in Figure 1. Furthermore, MCQs generation performance scores on the dataset were computed using precision and recall. Precision was to

validate the quality of the keywords by MCQs generation extracted from the lecture materials and recall was used to measure the quantity.

Precision answers the question – The keywords extracted from the lecture materials are they important words according to the gold standard marked by the teachers? While recall answers the question - are all the important words in the lecture materials extracted?

Table 3. Statistics of the Keywords automatically extracted from documents.

Lesson Materials	Number of Keywords Auto Extracted
Computer Science	368
Geology	318
Philosophy	254
Religion	488
History	471

For the performance evaluation, the following metrics precision, recall and f-measure are used in this research.

$$\text{Precision} = \frac{\text{numberCorrect}}{\text{numberExtracted}} \quad (1)$$

$$\text{Recall} = \frac{\text{numberCorrect}}{\text{totalExtracted}} \quad (2)$$

where numberCorrect is equal to true positive (TP), that is, important keywords extracted. numberExtracted in eqn 1 is summation of TP and false positive (FP). False Positive is extracted keywords that are not important words according to the gold standard. totalExtracted in eqn 2 is the summation of TP and false negative (FN). FN are keywords not extracted but are important words. To find the harmonic mean (average) of the precision and recall for computation of balance between precision (p) and recall (r) in the system:

$$\text{F - measure} = 2 * \left(\frac{\text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}} \right) \quad (3)$$

For each lecture material, for instance Computer Science (CS), there are two sets of data:

- Gold standard keywords that are important words in A prepared by the teacher. Henceforth known as truthset.
- The keywords extracted automatically from A. Henceforth known as extractedset.

From the above explanation, assuming truthset and extractedset are sets from CS. TP is a set of intersection between truthset and extractedset (TP = truthset & extractedset), FP is a set of keywords that are in extractedset but not in truthset (fp = extractedset – truthset), while FN is a set of keywords that are in truthset but not in extractedset (fn = truthset – extractedset). Using Table 1, there are 206 truthset and 368 extractedset for CS lesson material.

Table 4. Precision, Recall and F-measure.

Lesson Materials	TP	FP	FN	Precision	Recall	F-measure
Computer Science	162	206	31	0.44	0.84	0.58
Geology	79	293	3	0.25	0.96	0.39
Philosophy	94	160	2	0.37	0.98	0.54
Religion	169	319	1	0.35	0.94	0.51
History	185	286	18	0.39	0.91	0.55

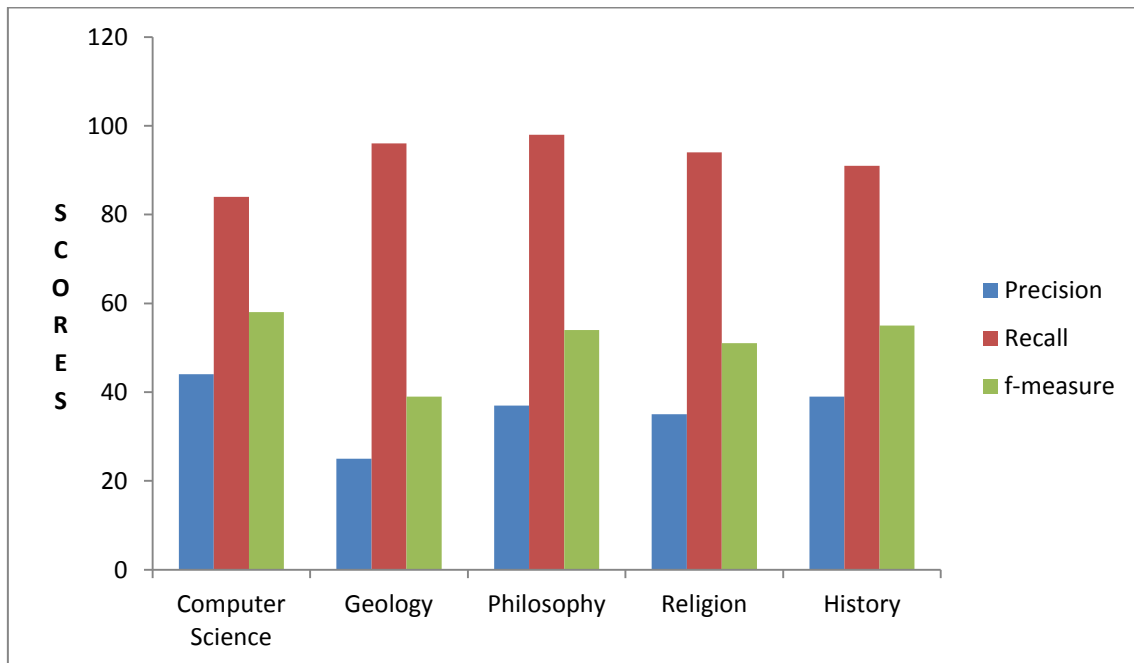


Figure 2. Bar charts of Precision, Recall and F-measure of scores on Table 4.

From Table 4 and Figure 2, it can be observed from the Recall scores that the majority of the keywords automatically extracted are important words. Although Precision scores show that there are other keywords automatically extracted that are not important according to the gold standard set. At the moment we want to ensure that the majority (if not all) the keywords extracted from lesson notes are in the gold standard sets. The keywords extracted by the NLP processes are used to generate Multi-Choice Questions (MCQs).

Since teachers are not technical persons, we develop a user-friendly interface where the MCQs generated are presented for teachers to easily access them. MCQs are generated from the extracted keywords by locating each keyword position given a document.

Table 5. Sample TF-IDF weightd sentences.

Sentence	TF-IDFWeights
The Difference Engine consisted entirely of mechanical components.	{'entirely': 0.305, 'consisted': 0.305, 'components': 0.305, 'mechanical': 0.209, 'difference': 0.209}

TEACHER'S CBT QUESTION BOARD

	Name: Onyedika Subjects: Contact: None Designation: None Session: 2019/2020 Class: Basic 7 Gold level: HIGHSCHOOL term: SECOND TERM	Enter your exam information Back to ANSWERS VIEW/DOWNLOAD BOARD Convert PDF to Text
---	---	--

Choose file

No file chosen

Upload

Enter the filename you just uploaded as is.

File name:

Generate Questions

Figure 3. Figure showing API for lecture material upload

question:

In 1936 , [Howard] Aiken had proposed his idea [to build a giant calculating machine] to the [Harvard University] Physics Department , ... He was told by the chairman , Frederick Saunders , that a lab _____ , Carmelo Lanza , had told him about a similar contraption already stored up in the Science Center attic .

options:**answer:****question:**

In 1936 , [Howard] Aiken had proposed his idea [to build a giant calculating machine] to the [Harvard University] Physics Department , ... He was told by the chairman , Frederick Saunders , that a lab technician , Carmelo Lanza , had told him about a similar contraption already stored up in the Science _____ attic .

options:**answer:**

Figure 4. Figure showing API for generated MCQs.

Each keyword of a document has a TF-IDF weight resulting in a composite-weighted sentence, this is illustrated in Table 5. How the weight in Table 5 work is when it is highest it means that t occurs many times within a small number of documents (thus lending high discriminating power to those documents); lower when the t occurs fewer times in a document, or occurs in many documents (thus offering a less pronounced relevance signal); lowest when the t occurs in virtually all documents. Given this definition, in a document we extract keywords with top weights as shown in Table 5. These extracted keywords are mapped to their positions within the document so that they are replaced with ‘-’ symbols instead. Then, we blindly and randomly select three other keywords from the pool of extracted keywords plus the main keyword that was replaced with ‘-’ to serve as options from which users are to select the right one. For a document, the number of multi-choice questions (MCQs) to be generated is dependent on the number of keywords extracted (see Table 5). These MCQs are regarded as suggested questions for the teachers to accept or reject. Figure 3 shows the API for lecture material upload while figure 4 show the API for the generated MCQs. The accept button is clicked by the teacher to select an MCQ if satisfied with it.

5. CONCLUSION

Multiple-choice questions (MCQs) generation for CBTE using Natural Language Processing (NLP) techniques is a system that helps teachers set multi-choice questions from their documents in a text file and thereafter provide answers to the questions generated. Consequently, it can be used in every area of education sector to test the knowledge or level of the students. However, this paper is not concerned with the issue of whether multiple-choice tests are better assessment methodology than other types of tests. What it focuses on is on application of NLP techniques to generate MCQs about facts explicitly based on keywords as stated in a text format of a given lesson material. From the results, it has shown that the system was capable of extracting keywords from lesson materials in setting examinable questions. This outcome is presented in a user-friendly interface for teachers easy accessibility using Django, a web framework for Python.

ACKNOWLEDGEMENTS

We thank all the teachers and students that contributed to the success of this work. Your feedbacks are very insightful and helpful towards the development of this work.

REFERENCES

- [1] Mannem Prashanth, Rashmi Prasad, and Aravind Joshi. (2010). Question generation from paragraphs at UPenn: QGSTEC system description. In Proceedings of QG2010: The Third Workshop on Question Generation, pages 84–91.
- [2] Colleen E Crangle and Joyce Brothers Kart. (2015). A questions-based investigation of consumer mental Health information. PeerJ, 3:e867.
- [3] Gall MD. (1970). The use of questions in teaching. Review of Educational Research, 40: 707–720.
- [4] Swart (2010). Evaluation of Final Examination Papers in Engineering: A Case Study Using Bloom’s Taxonomy. IEEE Transactions on Education, (May 2010) Vol. 53, No.2 257-264.
- [5] Aldabe, I., De Lacalle, M. L., Maritxalar, M., Martinez, E., and Uria, L. (2006). Arikiturri: an Automatic question generator based on corpora and nlp techniques. In International Conference on Intelligent Tutoring Systems (pp. 584-594). Springer, Berlin, Heidelberg.

- [6] Folajimi, Y. O., and Omojola, O. E. (2013). Natural language processing techniques for automatic test Questions generation using discourse connectives. *Journal of Computer Science and Its Application*, 20(2), 6076.
- [7] Aditya S. (2018). Using Natural Language Processing for Smart Question Generation. An Intel Developer Program Documentation. <https://software.intel.com/en-us/articles/using-natural-language-processing-for-smart-question-generation>.
- [8] Susanti, Y., Tokunaga, T., Nishikawa, H., and Obari, H. 2018. Automatic distractor generation for multiple Choice English vocabulary questions. *Research and Practice in Technology Enhanced Learning*, 13(1), 15.
- [9] Das, B., and Majumder, M. (2017). Factual open cloze question generation for assessment of learner's knowledge. *International Journal of Educational Technology in Higher Education*, 14(1), 24.

AI-Driven MCQ Generation Using NLP

Dr. Rajesh Saturi¹, Gorige Anusha², C. Sony³

¹Associate Professor, Computer Science and Engineering Vignana Bharathi Institute of Technology

^{2,3}B.Tech Honors Student, Computer Science and Engineering Vignana Bharathi Institute of Technology

Abstract—An AI-driven framework is proposed for automating the generation of Multiple-Choice Questions (MCQs) using Natural Language Processing (NLP), with the T5 model at its core. Traditional methods of MCQ creation are labor-intensive and depend heavily on subject-matter experts, limiting efficiency and scalability. This system addresses those limitations by leveraging transformer-based NLP models capable of processing educational materials in formats such as PDF, DOCX, and TXT. It generates context-aware questions and relevant distractors through modules focused on question generation, answer identification, and semantic similarity. The backend is implemented using FastAPI, while the user interface is built with React. Evaluation based on language quality, answer accuracy, and distractor relevance shows that the AI-generated MCQs closely match the quality of manually crafted ones, making the solution highly suitable for educational purposes.

Index Terms—AI-driven MCQ generation, Natural Language Processing, T5 Model, Text-to-Text, NLP-based Question Generation

1. INTRODUCTION

As digital learning continues to expand, the need for efficient and scalable assessment tools has become increasingly important. One key challenge in this space is the generation of Multiple-Choice Questions (MCQs), which traditionally demands significant time and subject knowledge from educators. This manual process limits adaptability, especially in dynamic or large-scale learning environments. To address this, recent developments in Natural Language Processing (NLP), particularly the use of

transformer-based models, offer promising solutions. The T5 model, known for its ability to reframe diverse NLP tasks into a unified text-to-text format, plays a central role in automating question generation. When combined with semantic search techniques for generating meaningful distractors and NLP-driven answer extraction methods, it becomes possible to automate the creation of high-quality, context-aware MCQs. This paper presents a modular, AI-based framework that brings these technologies

together in a streamlined pipeline. Users can upload PDF-based learning content, which is processed through layers of text parsing, concept identification, and question generation. The backend, built with Fast API, handles the processing logic, while a React-based frontend offers an accessible user experience. Designed to support both academic and professional learning scenarios, this system significantly reduces manual workload while ensuring the quality and relevance of the generated questions.

2. LITERATURE SURVEY

The automation of MCQ generation using NLP has been explored extensively in recent years, with various approaches offering their own methodologies, strengths, and limitations.

Ayushi Mathur and Dr. M. Suchithra proposed a method using abstractive summarization to simplify source content before MCQ creation. This method improved the linguistic fluency and coherence of questions, but it also posed the risk of omitting critical context from the original content (2024) [1].

Matthew Martianus Henry and Nur Adhianti Heryanto presented a broad literature review comparing rule-based, statistical, and deep learning methods for question generation. Their study offered valuable theoretical comparisons but lacked practical validation and empirical results (2024) [2].

Pochiraju and Chakilam introduced a hybrid approach that combines extractive summarization with XLNet for refining question context and accuracy. This ensured that factual content was retained, although the model relied heavily on redundant text and struggled with creativity in question formation (2023) [4].

Rao and Saha's two contributions include a survey of MCQ generation methods and a domain-specific model focused on school textbooks. Their survey categorized methods well, while the textbook model was effective for curriculum-aligned MCQs. However, both approaches were less flexible for broader or dynamic content inputs (2022) [5][7].

Altaj Virani and Rakesh Yadav developed a pipeline utilizing the T5 model for question generation and BERT for answer extraction. Although their method produced strong QA pairs, it did not address

distractor generation or UI deployment, limiting real-world applicability (2023) [6].

Hadifar and Bitew explored distractor reuse techniques by leveraging semantic similarity. This promoted diversity and efficiency but raised concerns about contextual relevance if not carefully validated (2024) [9].

Finally, Pawar and Dube implemented an end-to-end MCQ generation system using Langchain and Gemini LLM. Their model leveraged prompt engineering to generate dynamic MCQs but was resource-intensive and heavily dependent on precise prompt formulation (2022) [10].

3. PROPOSED SYSTEM ARCHITECTURE

The proposed system extracts text from uploaded documents of various PDF formats, including scanned files, text-based PDFs, and academic articles. It also supports other common document formats such as DOCX and plain TXT files, processes the extracted text, and generates multiple-choice questions (MCQs) with relevant answer choices. The backend is built using *FastAPI*, while the frontend is developed using *React*. The overall workflow consists of text extraction, MCQ generation, and final question refinement.

The structural diagram of an MCQ generation system outlines a sequential process. It begins with Text Acquisition, where the source material for questions is obtained. This is followed by Preprocessing, which likely involves cleaning and preparing the text for further analysis. Next, Question Generation algorithms create potential questions based on the processed text. Answer Selection identifies the correct answer for each generated question. The Distractor Generator then creates plausible but incorrect options. Finally, the MCQ Display presents the generated multiple-choice questions.

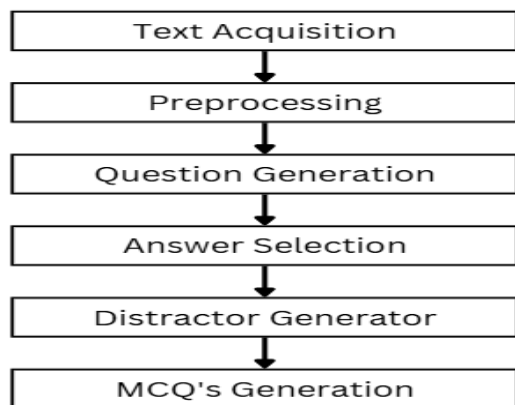


Figure 1: Structural Diagram of MCQ's Generation System

4. METHODOLOGY

The proposed system uses a multi-step process to automatically generate Multiple-Choice Questions (MCQs) from educational documents. It consists of two main components: a FastAPI-based backend for processing and a React frontend for user interaction. The backend handles file uploads, text extraction, and MCQ generation, while the frontend manages user inputs and displays the results.

4.1. Backend Implementation:

- FastAPI was chosen for the backend due to its high-performance and asynchronous capabilities. Cross-Origin Resource Sharing (CORS) middleware is added to enable communication between the frontend and backend. Uploaded files—such as PDFs, DOCX, or TXT—are processed using PyMuPDF to extract the text content. If the extraction fails or returns empty, the system notifies the user with an error message.
- After extracting the text, several NLP models process the content:
- Question Generation: The valhalla/t5-base-qg-hl model generates questions from key sentences.
- Answer Extraction: The deepset/roberta-base-squad2 model identifies accurate answers directly from the text.
- Distractor Generation: Using BAAI/bge-small-en-v1.5, a sentence-transformer model, the system finds distractors that are similar in context but incorrect, ensuring meaningful answer choices.

Once the MCQs are created, they undergo a validation phase. This includes grammar checks, difficulty evaluation, and proper formatting to ensure the questions meet quality standards. The backend also includes error handling for consistent performance at every stage.

4.2. Frontend Interface:

The frontend is developed using React, offering an intuitive user interface. Users can upload files and receive generated MCQs in a clean, readable format. React's state management (e.g., `useState`) is used to track the uploaded file, loading status, and server response.

- For file uploads, Axios sends a POST request with the document in multipart/form-data format to the FastAPI backend. The backend returns a structured response containing the question, correct answer, and distractors, which is then rendered dynamically on the screen. A loading spinner provides real-time feedback during processing.

- Supported file formats include PDF, DOCX, and TXT, ensuring broad usability across different content types.
- The internal pipeline is designed to ensure that the generated MCQs are accurate, contextually relevant, and educationally appropriate. The process includes:
 - Concept Identification: Using Named Entity Recognition (NER) and dependency parsing, the system identifies important terms and concepts from the text to guide question creation.
 - Question Generation: The T5-based model creates well-formed, grammatically correct questions based on the identified content, maintaining coherence with the original material.
 - Answer Selection: The Roberta-based model extracts exact answers from the text, ensuring they are precise and relevant.
 - Distractor Creation: Semantic similarity techniques are used to produce distractors that are related to the topic but clearly incorrect, avoiding ambiguity and ensuring fairness.
 - Validation and Formatting: A final review checks grammar, structure, and difficulty level. MCQs are then formatted for clarity and usability.

5. RESULTS AND DISCUSSION

5.1. Comparative Analysis

To evaluate the effectiveness of our proposed system, we compare it with the work titled "Automatic Multiple Choice Question Generation from Text: A Survey" by Dhawaleswar Rao CH and Sujun Kumar Saha (2022) [5]. Their survey compiles and categorizes various MCQ generation approaches, including rule-based, statistical, and neural models. While comprehensive, their work primarily focuses on theoretical exploration and categorization of methodologies without presenting a complete, practical implementation pipeline.

In contrast, our system offers a full-stack solution that includes both backend (FastAPI) and frontend (React) components. It supports diverse document formats such as PDF, DOCX, and TXT, and processes both structured and scanned files. We employ transformer-based models not only for question generation (T5) and answer extraction (RoBERTa) but also introduce semantic similarity via Sentence Transformers for distractor generation—something not covered in the surveyed models.

Table:Comparative Analysis

Criteria	Existing system [5]	Proposed System
Type of Work	Theoretical review	Practical end-to-end implementation
Question Generation	General neural model discussion	T5 transformer model
Answer Extraction	Not implemented	Statistical, Neural, RoBERTa (QA model)
Distractor Creation	Not covered in depth	Semantic search using Sentence Transformers
Evaluation	Qualitative	Qualitative (improved accuracy)

Unlike the survey, which evaluates techniques qualitatively, our system was evaluated through internal testing, showing promising results in generating questions that align well with expected educational outcomes. This highlights the practical viability and precision of our pipeline, which integrates question logic, validation layers, and user interactivity for real-time assessment creation.

The AI-driven MCQ generator demonstrates the ability to create structured and meaningful assessment questions. Performance evaluation is based on:

- Accuracy & Precision: Measures the relevance and correctness of generated MCQs.
- Semantic Analysis: Ensures logical consistency in question-answer pairs.
- User Feedback: Reviews from educators validate the effectiveness of automated MCQs.

MCQ Generator

No file chosen

6. CONCLUSION

This study presents an automated approach to MCQ generation using NLP techniques, particularly T5 and text-to-text transformations. By utilizing AI, the process of question generation is enhanced in terms of efficiency, scalability, and accuracy. Future work will focus on domain-specific customization and

improving the quality of generated distractors for broader applications in education and training.

7. FUTURE SCOPE

In the future, this system can be extended to generate questions from visual elements such as diagrams, flowcharts, and tables. Integrating image processing and OCR techniques can allow the extraction of visual context for question formation. Additionally, implementing adaptive difficulty levels by categorizing questions into easy, moderate, and advanced can enhance learner engagement and personalized assessment. These improvements would further broaden the applicability of the system across a variety of educational settings.

REFERENCES

- [1]. Pratik Pawar, Raghav Dube (2024). Automated Generation and Evaluation of Multiple-Choice Quizzes using Langchain and Gemini LLM.
- [2]. Matthew Martianus Henry, Nur Adhianti Heryanto (2024). Automatic Multiple Choice Question Generation: A Systematic Literature Review.
- [3]. V. Shobha Rani (2024). The Future of Assessment: Automating Multiple Choice Question Generation.
- [4]. Dhananjaya Pochiraju, Abhinav Chakilam (2023). Extractive Summarization and Multiple-Choice Question Generation using XLNet.
- [5]. Dhawaleswar Rao CH, Sujana Kumar Saha (2022). Automatic Multiple Choice Question Generation from Text: A Survey.
- [6]. Altaj Virani, Rakesh Yadav (2023). Automatic Question Answer Generation using T5 and NLP.
- [7]. Dhawaleswar Rao CH, Sujana Kumar Saha (2023). Generation of Multiple-Choice Questions from Textbook Contents of School-Level Subjects.
- [8]. Mohammed Ashraf Mohammed, Mostafa Ashraf Borhamy (2023). End-to-End Quiz-Style Question Generation for Educational Purposes.
- [9]. Amir Hadifar, Semere Kiros Bitew (2024). Learning to Reuse Distractors to Support Multiple-Choice Question Generation in Education.
- [10]. Ayushi Mathur, Dr. M. Suchithra (2022). Application of Abstractive Summarization in Multiple Choice Question Generation.
- [11]. Manning, C.D, and Schutze, H., 2007 "Foundations of Statistical Natural Language Processing" The MIT Press.
- [12]. Goyvaerts, P. 1996 "Controlled English, curse or blessing? A user perspective" Proceedings of the 1st International workshop on Controlled Language Applications (CLAW '96) (Leuven). 137-42 .
- [13]. Foster, R.M. 2009 "Improving the Output from Software that Generates Multiple Choice Question (MCQ) Test Items Automatically using Controlled Rhetorical Structure Theory" In proceedings of RANLP 2009, Borovets – Student Conference.
- [14]. Mitkov, R., and L. A. Ha. 2003. "Computer-Aided Generation of Multiple-Choice Tests." In Proceedings of HLT-NAACL 2003 Workshop on Building Educational Applications Using Natural Language Processing, pp. 17-22. Edmonton, Canada.
- [15]. Mitkov, R., L. A. Ha, and N. Karamanis. 2006. "A computer-aided environment for generating multiple-choice test items." *Natural Language Engineering* 12(2): 177-194.
- [16]. Reiter and R. Dale. 2000. "Building natural language generation systems." Cambridge University Press.
- [17]. Mitkov, R., 2004. "The Oxford Handbook of Computational Linguistics "Oxford University press
- [18]. Rodriguez-Torrealba, R., Garcia-Lopez, E., & Garcia-Cabot, A. (2022). End-to-end generation of multiple-choice questions using text-to-text transfer transformer models. *Expert Systems with Applications*, 208, 118258.
- [19]. Tami, M., Ashqar, H. I., & Elhenawy, M. (2024). Automated Question Generation for Science Tests in Arabic Language Using NLP Techniques. *arXiv preprint arXiv:2406.08520*.
- [20]. Vichare, S., Gawade, A., & Mangrulkar, R. (2024). Qgen: A Unique Question Generation and Answer Evaluation Technique Using Natural Language Processing. *Journal Transformations*, 38(1). of Engineering Education
- [21]. Ling, J., & Afzaal, M. (2024). Automatic question-answer pairs generation using pre-trained large language models in higher education. *Computers and Education: Artificial Intelligence*, 100252.
- [22]. Barlybayev, A., & Matkarimov, B. (2024). Development of system for generating

questions, answers, distractors using transformers. International Journal of Electrical & Computer Engineering (2088-8708), 14(2).

- [23]. Virani, A., Yadav, R., Sonawane, P., & Jawale, S. (2023, June). Automatic question answer generation using T5 and NLP. In 2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS) (pp. 1667-1673). IEEE.

An Automated MCQ Generator using NLP

Arpan Neupane¹; Sonam Chaudhari²; Suruchi Shah³

^{1,2,3}Dept. of Electronics and Computer Engineering Purwanchal Engineering Campus, IOE Dharan, Nepal

Publication Date: 2025/07/10

Abstract: The automation of multiple-choice question (MCQ) generation has emerged as a crucial advancement in educational assessment, aiming to reduce the time, effort, and domain expertise required for manual question creation. This research introduces a Natural Language Processing (NLP)-based system that generates high-quality, contextually relevant MCQs from textual content. The system accepts diverse input formats, including plain text and PDF documents, and utilizes advanced transformer models such as T5 (Text-to-Text Transfer Transformer), Flan-T5 (Fine-tuned Language Net T5), and DistilBERT (Distilled Bidirectional Encoder Representations from Transformers) for keyword extraction, question formulation, and distractor generation. A web-based interface, developed using Django, enables users to customize parameters like question quantity and model selection, ensuring flexibility across educational domains. Evaluation using BLEU (Bilingual Evaluation Understudy) and ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metrics confirms that fine-tuned models outperform their base counterparts in coherence and relevance. Additionally, the use of efficient fine-tuning techniques like LoRA (Low-Rank Adaptation) and QLoRA (Quantized Low-Rank Adaptation) significantly reduces computational overhead without degrading performance. The proposed system demonstrates strong potential to streamline formative assessment and enhance learning feedback loops, while future work will focus on mobile deployment and integration with digital learning platforms to expand accessibility.

Keywords: Automated MCQ Generation; NLP; Transformer Models; Distractor Generation; Educational Technology.

How to Cite: Arpan Neupane; Sonam Chaudhari; Suruchi Shah (2025). An Automated MCQ Generator using NLP. *International Journal of Innovative Science and Research Technology*, 10(6), 2941-2951. <https://doi.org/10.38124/ijisrt/25jun1878>

I. INTRODUCTION

MCQ (Multiple-choice questions) serve as fundamental assessment tools in modern education due to their efficiency in standardized testing and digital learning platforms. The growing demand for these assessments has highlighted the need for automated generation methods, as manual creation requires educators to carefully balance question clarity, answer plausibility, and content coverage, often demanding significant time investment. Recent advances in natural language processing have enabled various automated approaches to question generation, from early rule-based systems that proved inflexible across domains to more recent neural methods using transformer architectures that still face limitations in distractor quality and domain adaptation. This research investigates how hybrid architectures combining multiple natural language processing models can overcome current limitations in automated question generation. By systematically integrating the strengths of different approaches while addressing their weaknesses, we propose a transformer-based NLP (Natural Language Processing) solution that maintains academic rigor while offering practical flexibility for educators. Our approach specifically focuses on preserving question quality across diverse subject domains and input formats to meet real-world educational requirements, combining the generative capabilities of language models with specialized quality control mechanisms.

➤ Gap Identification:

Despite advancements in automated MCQ generation, several gaps remain. Many systems produce questions that are either conceptually weak or lack linguistic accuracy and contextual relevance. Customization options are often limited, restricting users from selecting preferred models or controlling the number of questions. Additionally, most tools cannot process diverse input formats like PDFs, reducing their practical applicability.

Recent studies emphasize key limitations in current distractor generation approaches. Chung et al. [6] observed that BERT-based models frequently generate distractors lacking semantic relevance, particularly in domain-specific areas such as biomedicine. Similarly, Grover et al. [5] reported that T5 models trained on general-purpose datasets often fail to capture subject-specific terminology and contextual nuances, resulting in distractors that are either too generic or contextually inappropriate. These limitations highlight the need for more domain-aware and semantically consistent methods in automated distractor generation.

➤ Objectives:

The main objectives of this system are as follows:

- To develop an NLP-based MCQ generation system that processes diverse inputs, including raw text and PDFs.

- To Provide customizable features for PDF conversion, question count selection, and model choice.

II. LITERATURE REVIEW

➤ *Related Work:*

Several studies have explored automatic MCQ generation using NLP techniques. Folajimi and Omojola [1] utilized discourse connectives, words indicating logical relationships between sentences, to identify meaningful question candidates. Their system processed user-provided text to generate relevant questions using NLP methods. Nwafor and Onyenwe [2] proposed a three-step MCQ generation strategy: sentence selection, gap selection, and question formation, aiming to improve logical coherence in generated items. Susanti et al. [3] introduced a context-aware method for distractor generation in vocabulary-based MCQs by ranking candidates using semantic similarity and collocation measures. Das and Majumder [4] focused on factual cloze question generation using POS (Part-of-Speech) tagging rules to extract informative sentences and create fill in the blank questions, also providing hints to guide learner responses. Grover et al. [5] demonstrated the effectiveness of fine-tuned T5 (Text-to-Text Transfer Transformer) for question generation, showing improved relevance and quality over rule based methods. Chung et al. [6] developed a BERT (Bidirectional Encoder Representations from Transformers) based distractor generation model that applied multitask learning and negative answer training to produce plausible yet incorrect distractors, enhancing MCQ challenge and quality.

➤ *Related Theory*

In this project, the exploration of related theories serves as a crucial foundation for the development and implementation of the MCQ generation. The following topics encapsulate the diverse theoretical frameworks that guide various aspects of the project:

➤ *Natural Language Processing:*

Natural Language Processing (NLP) is a branch of AI that enables computers to understand and generate human language. It involves tasks like tokenization and is used in applications such as text generation, summarization, and question answering.

➤ *Multiple Choice Question Generation:*

MCQ generation involves automatically creating well-structured questions and answer choices from text. This project uses models like T5, BERT, and BART (Bidirectional and Auto-Regressive Transformers) for question generation, RAKE and DistilBERT (Distilled Bidirectional Encoder Representations from Transformers) for keyword extraction, and Sense2Vec (Sense-aware Word2Vec) and LLaMa (Large Language Model Meta AI) for distractor generation. The key challenge is ensuring clarity, accuracy, and relevance.

➤ *Language Models:*

The different language models used in this project are:

• *T5 Base Model:*

Google's T5-Base is a text-to-text transformer that reformulates NLP tasks as text generation, making it effective for MCQ creation. Using an encoder-decoder architecture, it processes input passages and generates grammatically correct, context-aware questions and answer choices. Fine-tuned on QA (Question Answering) datasets like SQuAD v2 (Stanford Question Answering Dataset version 2), it produces well-structured, high-quality MCQs automatically.

• *FLAN-T5 Model (Fine Tuned Language Net T5):*

Flan-T5-Small is a fine-tuned, instruction-optimized variant of T5 designed for better task understanding and efficiency. With the same encoder-decoder setup but smaller size, it generates coherent, contextually relevant MCQs from structured prompts, making it ideal for automated educational assessments.

• *DistilBERT:*

DistilBERT's architecture for MCQ generation tokenizes input text and uses a transformer encoder to capture contextual relationships, producing rich semantic embeddings. It generates questions by identifying key concepts and creates distractors via masked token prediction of plausible but incorrect options. Lightweight yet powerful, DistilBERT offers fast, accurate MCQ generation. Fine-tuned classifiers validate output quality. Integrated with Pandas and PyTorch, it enables an efficient, automated MCQ system.

• *LLaMa:*

LLaMa is a large-scale transformer model fine-tuned for instruction-following tasks, making it ideal for MCQ generation, keyword extraction, and distractor creation. It identifies key concepts to form clear questions, extracts relevant keywords, and generates plausible, context-aware distractors. Its advanced language understanding enables accurate, high-quality automated question creation.

• *BART:*

BART, a transformer model by Facebook, excels in text-to-text tasks like summarization and question generation. Its encoder-decoder design corrupts and reconstructs input to produce coherent, context-relevant MCQ stems, answer choices, and distractors. Pretrained on large datasets and fine-tuned on QA data, BART delivers high-quality questions. Integrated with Pandas and TensorFlow, it enables an efficient, automated MCQ generation pipeline for educational use.

➤ *Traditional NLP Techniques and Word Vector Models:*

• *RAKE (Rapid Automatic Keyword Extraction):*

RAKE (Rapid Automatic Keyword Extraction) is an unsupervised keyword extraction algorithm that identifies key terms by analyzing word co-occurrence patterns and frequency. It efficiently ranks candidate keywords without requiring training data, enabling automated generation of relevant MCQ stems and distractors through meaningful

concept extraction. This lightweight approach supports context-aware question creation.

- *spaCy*:

spaCy is used in MCQ generation for efficient keyword extraction. It leverages NLP techniques like dependency parsing and named entity recognition to identify key terms from text. These keywords help create relevant question stems, options, and distractors. spaCy's speed and accuracy make it ideal for scalable, content-focused question generation

- *Sense2Vec*:

Sense2Vec is an advanced word embedding model that enhances distractor generation by considering word senses, part-of-speech tags, and entity types. It helps create context-aware distractors that are semantically similar but meaningfully distinct, making MCQs more challenging and less predictable.

- *WordNet*:

WordNet is a lexical database used in MCQ generation to create meaningful and challenging distractors. It provides synonyms, antonyms, hypernyms, and hyponyms, helping generate context-aware, semantically relevant options that are close but incorrect. By organizing words into synsets, WordNet ensures distractors are diverse, accurate, and not too obvious or unrelated.

III. METHODOLOGY

The system accepts raw text or PDF uploads, extracting text automatically from PDFs. Inputs over 500 tokens are chunked for efficiency. Keywords are extracted using methods like spaCy, RAKE, or DistilBERT, which guide transformer models to generate questions and distractors. Duplicate distractors are removed in post-processing. Final MCQs are saved in a database for authenticated users or in session storage for guests.

➤ *Data Collection and Model Fine-Tuning:*

In this study, relevant datasets were collected from diverse sources to fine tune models for tasks including question generation, keyword extraction, and distractor generation. For question generation, the T5 base model was fine tuned using the SQuAD v2 dataset, comprising approximately 130,000 question answer pairs, to enhance its ability to generate contextually appropriate and well formed questions. Additionally, the Flan T5 Small model was fine tuned on several domain specific datasets, namely ScienceQA, General Knowledge, and Biomedical QA datasets, to improve performance in generating accurate and domain relevant multiple choice questions (MCQs). Keyword extraction leveraged a hybrid approach combining RAKE, spaCy, and a DistilBERT model fine tuned on the SciQ dataset, which contains roughly 13,000 entries. For distractor generation, LLM T5 and Llama3.2 models were fine tuned on the RACE dataset, consisting of 27,000 instances, alongside supplementary scientific datasets from Hugging Face, focusing on producing plausible distractors.

➤ *System design:*

The Schematic Diagram of the developed system is given in the figure 1:

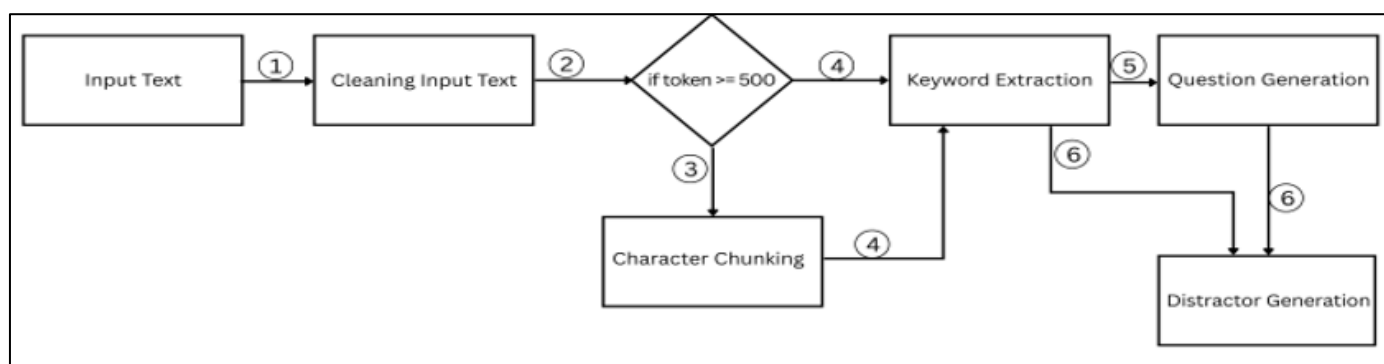


Fig 1 System Block Diagram

- *Input text Cleaning:*

The preprocessing of the input text is carried out using pandas, and all unnecessary elements such as symbols, HTML tags, numbering, bullets, and arrows are removed from the text. This leaves only textual content that is of importance to process further.

- *Tokenization and Chunking:*

If the input exceeds 500 tokens (approximately 2500 characters), it is segmented into smaller chunks to ensure effective processing by NLP models while preserving

contextual coherence. For shorter inputs, keyword extraction is performed directly without chunking.

- *Keyword Extraction:*

After the text is divided through chunking, keyword extraction is carried out to identify the most significant terms required for effective question generation. This process is performed using RAKE and spaCy, which incorporates a BART model fine-tuned for keyword identification. Additionally, a DistilBERT model, fine-tuned via QLoRA on science-specific datasets, is utilized to further improve the contextual relevance and accuracy of the extracted keywords

- **Question Generation:**

Question generation follows the keyword extraction phase. The system employs two models: T5-base, fine-tuned on the SQuAD v2 dataset, for general-purpose question generation, and Flan-T5 Small, adapted using domain-specific datasets such as ScienceQA, GeneralKnowledge, and Biomedical QA. These models utilize the extracted keywords and context to produce coherent and structurally sound questions.

- **Distractor Generation:**

After question formulation, distractors (incorrect options) are generated using a combination of Sense2Vec and WordNet. Sense2Vec first suggests candidates based on word similarity; if it fails, the system falls back on WordNet for synonyms. These methods are combined to ensure at least three distractors. Additionally, (*Large Language Model based on T5 architecture*) LLM-T5 (fine-tuned on the RACE dataset) and LLaMA (LoRA-tuned on science-domain data using the Unsloth platform) generate context-aware distractors based on the keyword and question, ensuring they are both plausible and challenging.

➤ **Selection of Transformer Models for MCQ Generation**

The choice of T5 and Flan T5 over large generative models such as GPT 3 or GPT 4 was due to their unified text-to-text framework, which is better suited for producing structured outputs like multiple-choice questions. While GPT models excel in open-ended generation, they often need postprocessing to fit MCQ formats, such as enforcing fixed answer choices. In contrast, the encoder-decoder architecture of T5 allows direct fine-tuning on task-specific templates, enhancing control and reducing variability. Flan T5 further improves this through instruction-based fine-tuning, enabling efficient domain adaptation without the high computational cost of larger language models.

➤ **Tools and Technique:**

- **Python:**

The core programming language used for algorithm implementation and backend development due to its simplicity and rich ecosystem.

- **Pandas:**

Used for data manipulation and preprocessing tasks, aiding in the handling of textual data before feeding it into NLP models.

- **PyTorch:**

Utilized for building and training NLP models, leveraging dynamic computation graphs and GPU acceleration

- **Hugging Face:**

Provided pre-trained NLP models and libraries critical for natural language understanding and question generation.

- **QLoRA(Quantized Low-Rank Adaptation) :**

A memory-efficient fine-tuning method utilizing 4-bit quantization and adapter layers to enable large model training on limited hardware while maintaining accuracy.

- **LoRA(Low-Rank Adaptation):**

A parameter-efficient fine-tuning technique that adds small trainable layers to large models, reducing resource requirements and accelerating training without compromising performance

- **Django:**

A high-level Python web framework used to develop the web backend, enabling rapid development and secure management of the application.

- **HTML, CSS, and JavaScript:**

Technologies used to create the responsive and interactive web frontend, facilitating user interaction such as MCQ generation and PDF uploads.

➤ **Evaluation Metrics:**

To assess the performance of the MCQ generation system, both automatic and training-based evaluation metrics were employed. These metrics help evaluate the contextual accuracy, structure, and relevance of the generated questions.

- **BLEU (Bilingual Evaluation Understudy):**

BLEU is an automated metric used to evaluate the quality of generated MCQ stems by comparing them to human-written references. It calculates the n-gram overlap (precision) between generated and reference texts. While a higher BLEU score indicates closer similarity, it does not fully capture semantic accuracy or creativity and is best complemented with human evaluation.

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \cdot \log p_n \right)$$

Where:

- P_n = Modified n -gram precision, calculated for n ranging from 1 to N (commonly up to 4), representing the proportion of n -gram overlaps between the generated and reference texts.
- W_n = weights for each n -gram
- BP (Brevity Penalty) = A factor introduced to penalize excessively short candidate outputs, defined as:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases}$$

Where:

- C = Length of the candidate (generated) sentence.
- R = Length of the reference sentence.

• *ROUGE (Recall-Oriented Understudy for Gisting Evaluation):*

ROUGE evaluates the quality of generated text by measuring overlap with reference text, focusing on recall. In MCQ generation, it helps assess whether the question stem captures key information. However, it mainly considers word overlap and may not fully reflect grammatical accuracy or distractor quality.

$$\text{ROUGE-N} = \frac{\sum_{\text{ref} \in \text{Refs}} \sum_{\text{gram}_n \in \text{ref}} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{\text{ref} \in \text{Refs}} \sum_{\text{gram}_n \in \text{ref}} \text{Count}(\text{gram}_n)}$$

Where:

- Gram_n = Represents an n -gram, such as unigrams (single words), bigrams (two-word sequences), etc
- $\text{Count}_{\text{match}}(\text{gram}_n)$ = Denotes the number of matching n -grams between the generated output and the reference text.
- Refs = Refers to the set of reference texts used for comparison.

Table 1 System Specification Table

S. N	Particulars	System Specifications
1	GPU Type	RTX4090
2	GPU Size	32 GB
3	RAM size	32 GB

➤ *Training and Evaluation of T5 Base Model:*

• *Model Training:*

For the Training specification of T5 base, the parameter required were configured as mentioned below in Table 2

Table 2 Training Specification Table for T5 base model

S. N	Particulars	Applied Specifications
1	Number of Epochs	7
2	Batch Size	16
3	Optimizer	AdamW
4	Learning Rate	3e-10
5	Momentum	0.9
6	Decay	0.01

• *Training Loss:*

Training loss indicates how well the model learns from the training data. Calculated using loss functions like cross-entropy, lower values signify better alignment with expected outputs

• *Validation Loss:*

Validation Loss evaluates model performance on unseen data. A low validation loss implies strong generalization, while a significant gap between training and validation loss may suggest overfitting.

IV. RESULT AND DISCUSSION

➤ *System Overview:*

The MCQ Generator effectively processes input text to extract keywords, generate questions, and produce relevant distractors. T5-base performed well for general questions, while Flan-T5 Small improved accuracy on science content. RAKE, spaCy, and DistilBERT enabled robust keyword extraction, and Sense2Vec, LLM-T5, and LLaMA enhanced distractor quality.

For the system specification, the parameter required were configured as mentioned below in Table 1.

- *Evaluation:*

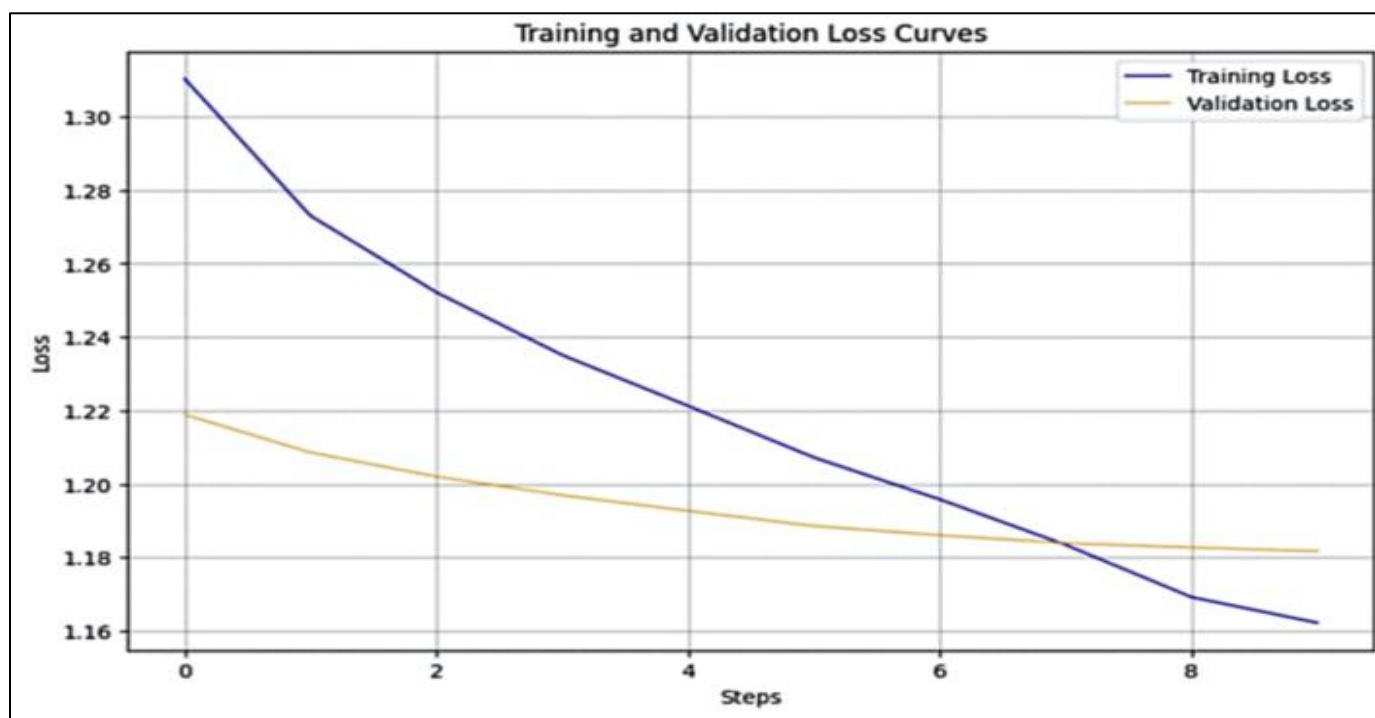


Fig 2 Training vs Validation Loss in T5 Base Mod

Fig 2 shows the training and validation loss for the T5-base model. While both losses decrease over time, the training loss drops more rapidly, indicating effective learning.

The slower decline and consistent gap in validation loss suggest some overfitting and room for improved generalization.

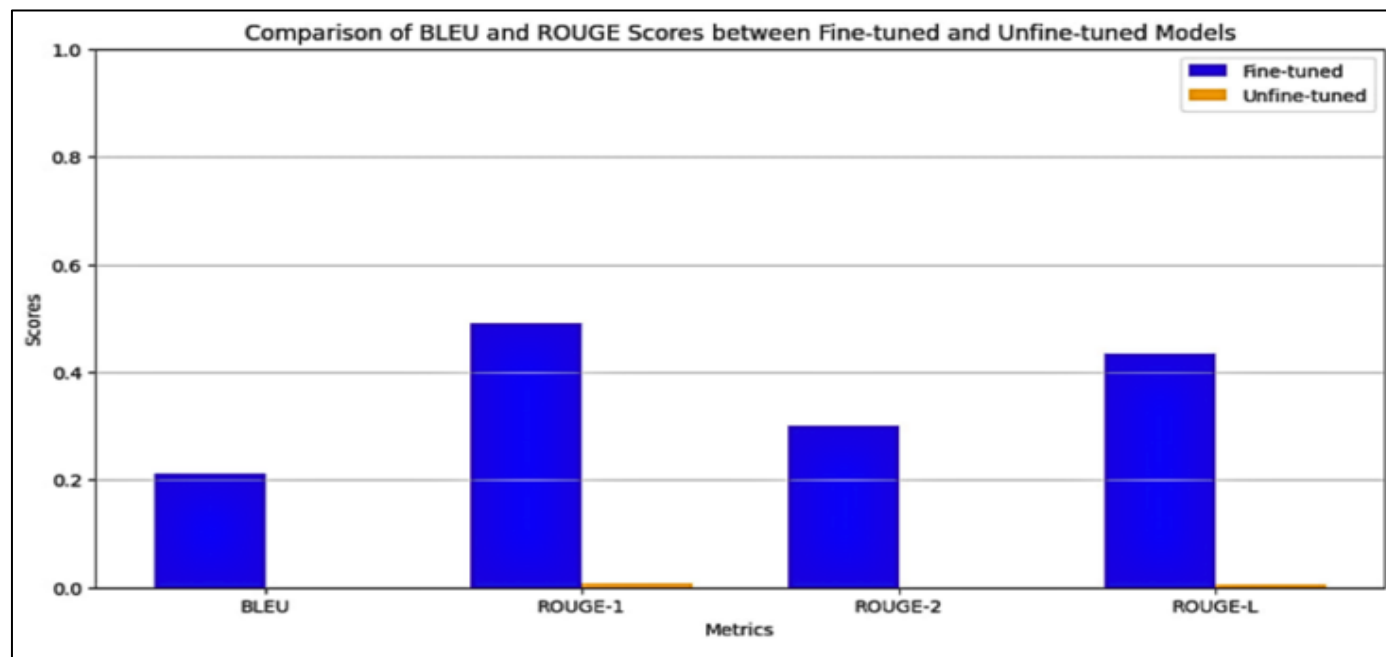


Fig 3 Comparison of BLEU and ROUGE Scores between Fine-Tuned and Unfine-Tuned Model in T5 Base Model

Fig 3 compares BLEU and ROUGE scores between fine-tuned and unfine-tuned T5-base models. The fine-tuned model significantly outperforms the baseline, achieving a BLEU score of 0.22 versus near-zero for the unfine-tuned version. ROUGE-1, ROUGE-2, and ROUGE-L scores also improve markedly to 0.48, 0.32, and 0.44, compared to 0.02, 0.005, and 0.015, respectively. These results highlight the

impact of fine-tuning in enhancing text generation quality, coherence, and contextual relevance for MCQ generation.

➤ *Training and Evaluation of Flan-T5 Small Model:*

- *Model Training*

For the Training specification of Flan-T5 small model, the parameter required were configured as mentioned below in Table 3

Table 3 Training Specification Table for Flan-T5 small model

S. N	Particulars	Applied Specifications
1	Number of Epochs	10
2	Batch Size	16
3	Optimizer	AdamW
4	Learning Rate	5e-10
5	Momentum	0.9
6	Decay	0.01

• *Evaluation:*



Fig 4 Training vs Validation loss in Flan-T5 small model

Fig 4 illustrates the behavior of training loss and validation Flan-T5 during the model training phase. The training loss begins higher than the validation loss (orange line) and decreases over time, though it exhibits fluctuations, indicating some instability in the training process. In contrast, the validation loss decreases smoothly, reflecting the model's ability to improve its generalization to unseen data.

Throughout the process, the training loss remains consistently lower than the validation loss, signaling a degree of overfitting. The observed fluctuations in the training loss could be attributed to factors such as an inconsistent learning rate or noisy data. A smaller and more stable gap between the two losses would suggest better generalization performance.

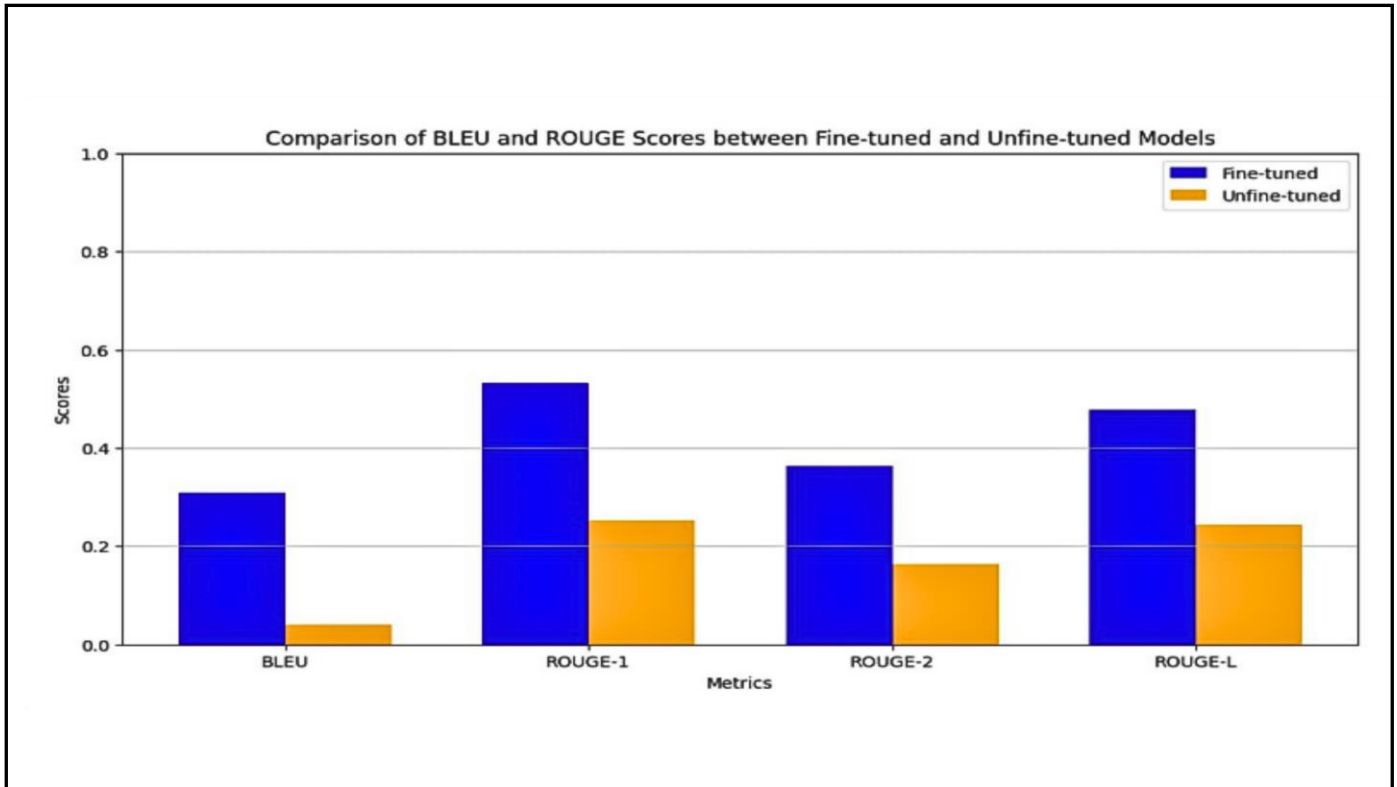


Fig 5 Comparison of BLEU and ROUGE Scores between Fine-Tuned and Unfine-Tuned Model in Flan-T5 Small Model

Fig 5 compares BLEU and ROUGE scores between fine-tuned and unfine-tuned Flan T5 Small models. The fine-tuned model achieves higher scores across all metrics, with a BLEU score of 0.28 versus 0.05 for the unfine-tuned version. ROUGE-1, ROUGE-2, and ROUGE-L scores also improve to 0.52, 0.35, and 0.47, compared to 0.26, 0.18, and 0.27, respectively. These results confirm that fine-tuning

significantly improves text similarity, coherence, and contextual relevance in question generation.

➤ *Evaluation of LLM DistilBERT and LLM-T5 Distractor:*

- *LLM DistilBERT:*

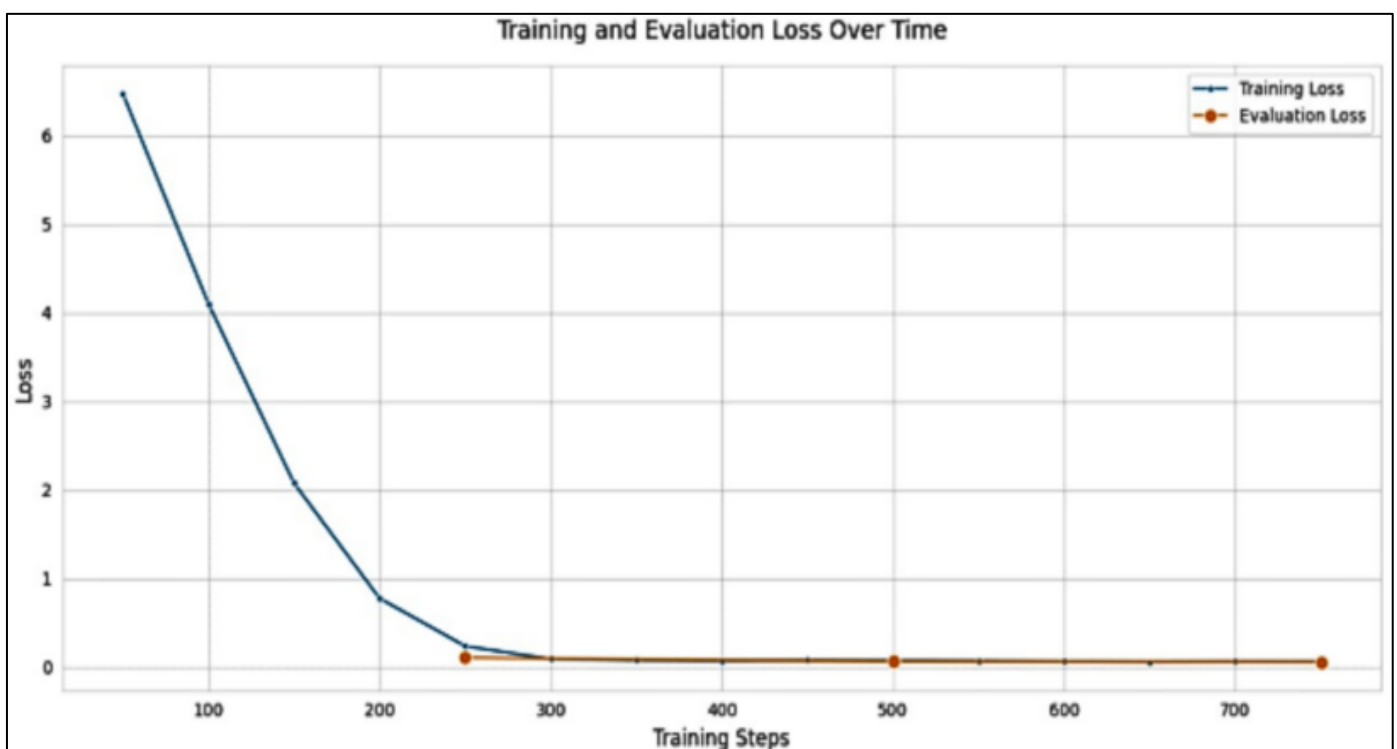


Fig 6 Training and Validation Loss in LLM DistilBERT

Fig 6 shows a sharp decline in training loss, stabilizing near zero after step 300, while evaluation loss remains consistently low. The close alignment of both indicates effective learning with minimal overfitting and strong generalization.

- *LLM-T5 Distractor*

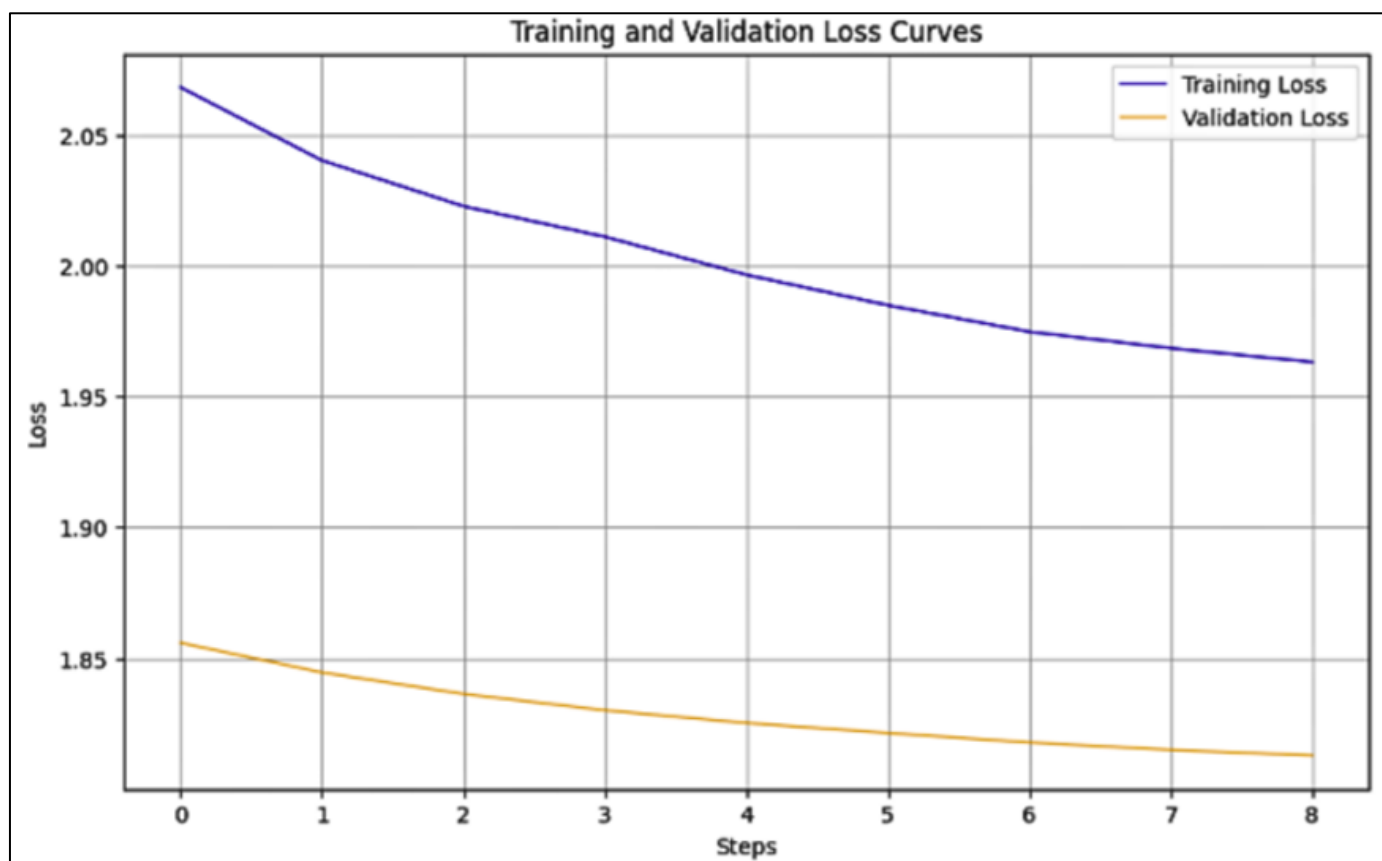


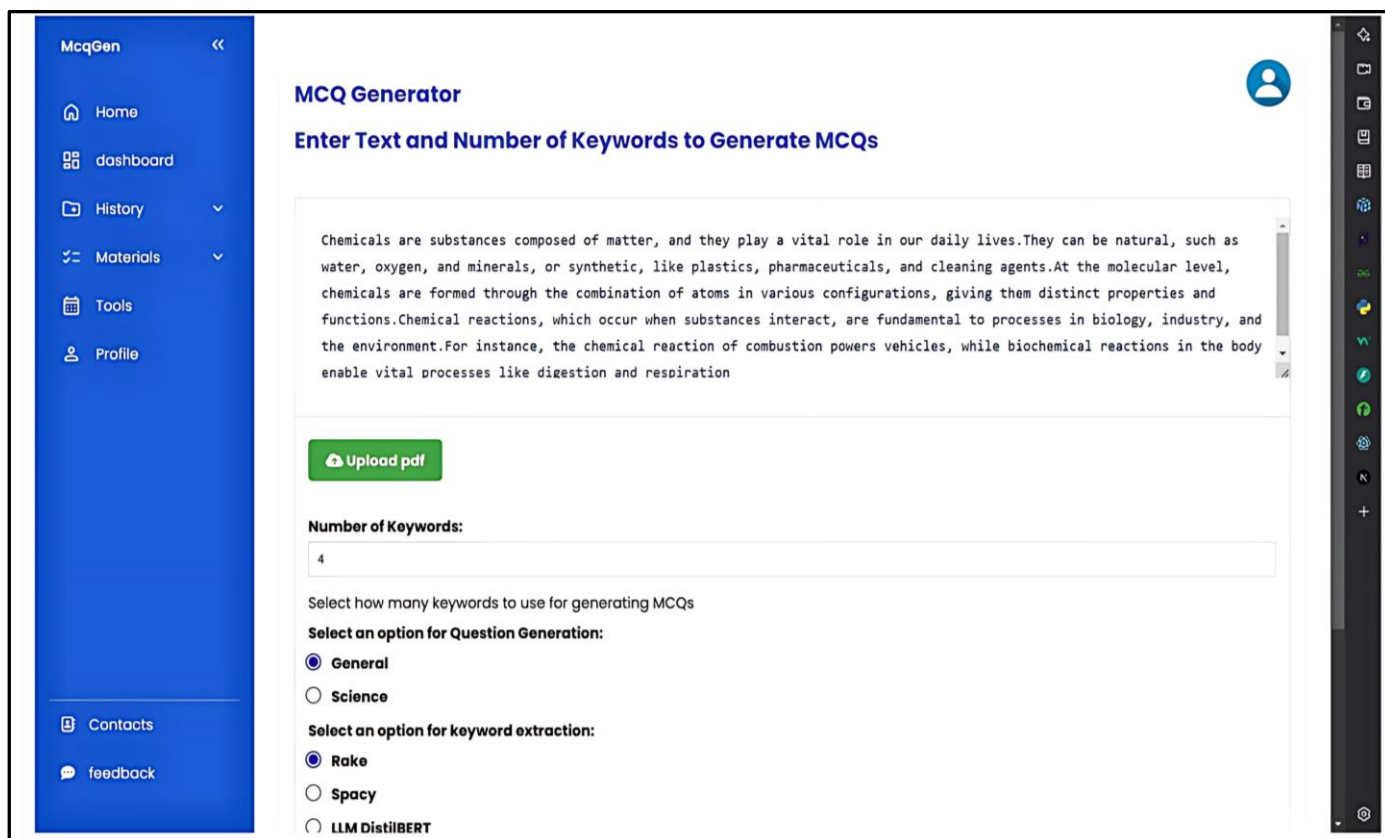
Fig 7 Training and Validation Loss in LLM-T5 Distractor Generator

Figure 7 shows the training and validation loss curves for the LLM-T5 distractor generator. The training loss drops sharply initially and stabilizes near zero by step 300, indicating efficient learning. The validation loss remains consistently low, closely tracking the training loss, which

suggests strong generalization without overfitting. This demonstrates the model's robustness and effective training.

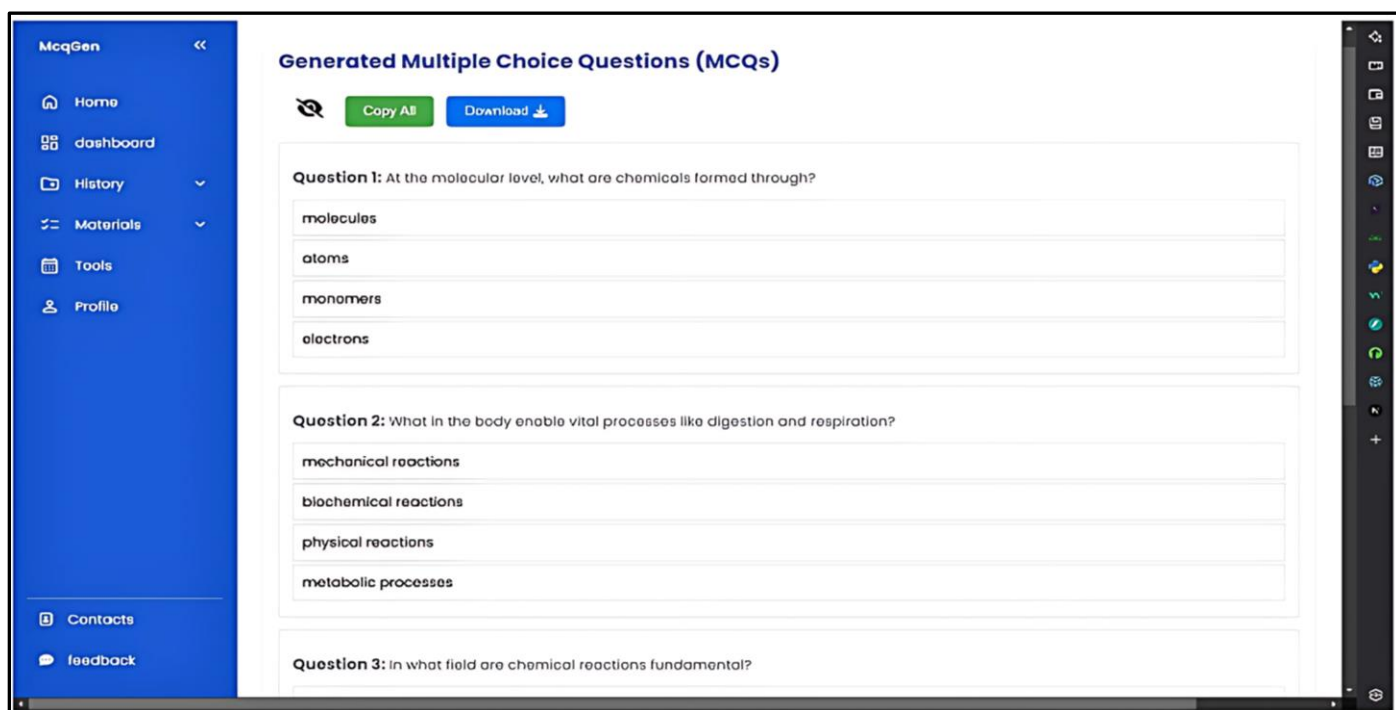
➤ *Final Output:*

The user interface of the webapp is shown in Figure 8 and Figure 9.



The screenshot shows the 'MCQ Generator' web application. On the left is a blue sidebar with navigation links: Home, dashboard, History, Materials, Tools, Profile, Contacts, and feedback. The main content area is titled 'MCQ Generator' and 'Enter Text and Number of Keywords to Generate MCQs'. It features a text input field containing a paragraph about chemicals. Below the text is an 'Upload pdf' button. Further down, there is a 'Number of Keywords' input field set to '4'. Below this, there are two sections for selection: 'Select an option for Question Generation' with radio buttons for 'General' (selected) and 'Science'; and 'Select an option for keyword extraction' with radio buttons for 'Rake' (selected), 'Spacy', and 'LLM DistilBERT'. A user profile icon is visible in the top right corner.

Fig 8 Text Input and Model Selection in System



The screenshot shows the 'Generated Multiple Choice Questions (MCQs)' section of the application. It includes 'Copy All' and 'Download' buttons. Three questions are displayed, each with four multiple-choice options in a scrollable list:

- Question 1:** At the molecular level, what are chemicals formed through?
 - molecules
 - atoms
 - monomers
 - ol electrons
- Question 2:** What in the body enable vital processes like digestion and respiration?
 - mechanical reactions
 - biochemical reactions
 - physical reactions
 - metabolic processes
- Question 3:** In what field are chemical reactions fundamental?

Fig 9 Generated MCQ

➤ **Limitations:**

- The system performs optimally in STEM subjects but shows limited effectiveness in humanities, where subjective interpretation and cultural context are critical.
- Performance is highly sensitive to input quality; noisy PDFs or unstructured text reduce the accuracy of question generation.
- The models are trained primarily on English-language datasets, limiting their applicability in non-English or multilingual educational contexts.

- Despite the use of LoRA and QLoRA for optimization, fine-tuning models for specialized domains such as law or medicine remains computationally intensive.

V. CONCLUSION

In conclusion this work addressed the challenge of automating MCQ generation, a traditionally time-consuming and expertise-driven task in education. By leveraging NLP techniques and fine-tuned transformer models, the system effectively processes input text or PDFs to generate contextually relevant questions and plausible distractors. Fine-tuning models like T5 and Flan-T5 on domain-specific datasets significantly improved question quality, while hybrid distractor generation using Sense2Vec, WordNet, and LLMs ensured relevance and variety. Evaluation metrics demonstrated clear performance gains, validating the effectiveness of the approach. The resulting Django-based application offers a flexible, scalable, and user-friendly solution for automated MCQ generation.

➤ Future Enhancements:

- Develop a lightweight mobile app that works offline, enabling students and teachers to generate and practice MCQs anytime, anywhere.
- Expand the project into an online exam platform with teacher-admin quizzes, student login, automatic grading, performance analytics, and adaptive question banks.
- Extend the system's capabilities by fine-tuning models on multilingual and humanities-focused datasets to improve performance in non-STEM domains that require nuanced understanding of language, culture, and subjectivity.

ACKNOWLEDGMENT

The authors would like to express sincere gratitude to Mr. Manoj Kumar Guragain, Head of the Electronics and Computer Engineering Department (IOE Purwanchal Campus) for his invaluable guidance and insightful feedback. The authors would also like to express gratitude towards the Electronics and Computer Engineering Department (IOE Purwanchal Campus), associated teachers, and all the well-wishers.

The authors take full responsibility for any errors or omissions that may have occurred in preparing this research paper and would like to welcome comments and queries.

REFERENCES

- [1]. Y. Folajimi and O. Omojola, "Natural language processing techniques for automatic test questions generation using discourse connectives," *Journal of Computer Science and Its Application*, vol. 20, no. 2, pp. 60–76, 2013.
- [2]. Nwafor and I. E. Onyenwe, "An automated multiple-choice question generation using natural language processing techniques," *arXiv preprint*, arXiv:2103.14757, 2021.
- [3]. Y. Susanti, T. Tokunaga, H. Nishikawa, and H. Obari, "Automatic distractor generation for multiple-choice English vocabulary questions," *Research and Practice in Technology Enhanced Learning*, vol. 13, no. 1, p. 15, 2018.
- [4]. Das and M. Majumder, "Factual open cloze question generation for assessment of learner's knowledge," *International Journal of Educational Technology in Higher Education*, vol. 14, pp. 1–12, 2017.
- [5]. K. Grover, K. Kaur, K. Tiwari, Rupali, and P. Kumar, "Deep learning based question generation using T5 transformer," in *Advanced Computing: 10th International Conference, IACC 2020, Panaji, Goa, India, December 5–6, 2020, Revised Selected Papers, Part I*, Springer, 2021, pp. 243–255.
- [6]. H.-L. Chung, Y.-H. Chan, and Y.-C. Fan, "A BERT-based distractor generation scheme with multi-tasking and negative answer training strategies," *arXiv preprint*, arXiv:2010.05384, 2020.
- [7]. H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziere, N. Goyal, E. Hambro, F. Azhar, and others, "LLaMA: Open and efficient foundation language models," *arXiv preprint*, arXiv:2302.13971, 2023.
- [8]. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.
- [9]. V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," *arXiv preprint*, arXiv:1910.01108, 2019.
- [10]. J. Oza and H. Yadav, "Enhancing question prediction with FLAN-T5: A context-aware language model approach," *Authorea Preprints*, 2023.
- [11]. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [12]. N. C. P. Prakash, A. P. Narasimhaiah, J. B. Nagaraj, P. K. Pareek, N. B. Maruthikumar, and R. I. Manjunath, "Implementation of NLP based automatic text summarization using spaCy," *International Journal of Health Sciences*, vol. 6, no. S5, pp. 7508–7521, 2022.
- [13]. S. Rose, D. Engel, N. Cramer, and W. Cowley, "Automatic keyword extraction from individual documents," in *Text Mining: Applications and Theory*, pp. 1–20, 2010.

Know What You Don’t Know: Unanswerable Questions for SQuAD

Pranav Rajpurkar* Robin Jia* Percy Liang

Computer Science Department, Stanford University

{pranavs, robinjia, pliang}@cs.stanford.edu

Abstract

Extractive reading comprehension systems can often locate the correct answer to a question in a context document, but they also tend to make unreliable guesses on questions for which the correct answer is not stated in the context. Existing datasets either focus exclusively on answerable questions, or use automatically generated unanswerable questions that are easy to identify. To address these weaknesses, we present SQUADRUN, a new dataset that combines the existing Stanford Question Answering Dataset (SQuAD) with over 50,000 unanswerable questions written adversarially by crowdworkers to look similar to answerable ones. To do well on SQUADRUN, systems must not only answer questions when possible, but also determine when no answer is supported by the paragraph and abstain from answering. SQUADRUN is a challenging natural language understanding task for existing models: a strong neural system that gets 86% F1 on SQuAD achieves only 66% F1 on SQUADRUN. We release SQUADRUN to the community as the successor to SQuAD.

1 Introduction

Machine reading comprehension has become a central task in natural language understanding, fueled by the creation of many large-scale datasets (Hermann et al., 2015; Hewlett et al., 2016; Rajpurkar et al., 2016; Nguyen et al., 2016; Trischler et al., 2017; Joshi et al., 2017). In turn, these datasets have spurred a diverse array of model architecture improvements (Seo et al., 2016; Hu

Article: Endangered Species Act

Paragraph: “... Other legislation followed, including the Migratory Bird Conservation Act of 1929, a 1937 treaty prohibiting the hunting of right and gray whales, and the Bald Eagle Protection Act of 1940. These later laws had a low cost to society—the species were relatively rare—and little opposition was raised.”

Question 1: “Which laws faced significant opposition?”

Plausible Answer: later laws

Question 2: “What was the name of the 1937 treaty?”

Plausible Answer: Bald Eagle Protection Act

Figure 1: Two unanswerable questions written by crowdworkers, along with plausible (but incorrect) answers. Relevant keywords are shown in blue.

et al., 2017; Wang et al., 2017; Clark and Gardner, 2017; Huang et al., 2018). Recent work has even produced systems that surpass human-level exact match accuracy on the Stanford Question Answering Dataset (SQuAD), one of the most widely-used reading comprehension benchmarks (Rajpurkar et al., 2016).

Nonetheless, these systems are still far from true language understanding. Recent analysis shows that models can do well at SQuAD by learning context and type-matching heuristics (Weissenborn et al., 2017), and that success on SQuAD does not ensure robustness to distracting sentences (Jia and Liang, 2017). One root cause of these problems is SQuAD’s focus on questions for which a correct answer is guaranteed to exist in the context document. Therefore, models only need to select the span that seems most related to the question, instead of checking that the answer is actually entailed by the text.

In this work, we construct SQUADRUN,¹ a new dataset that combines the existing questions in SQuAD with 53,775 new, unanswerable ques-

* The first two authors contributed equally to this paper.

¹ SQuAD with adversarial Unanswerable questions

tions about the same paragraphs. Crowdworkers crafted these questions so that (1) they are *relevant* to the paragraph, and (2) the paragraph contains a *plausible answer*—something of the same type as what the question asks for. Two such examples are shown in Figure 1.

We confirm that SQUADRUN is both challenging and high-quality. A state-of-the-art model achieves only 66.3% F1 score when trained and tested on SQUADRUN, whereas human accuracy is 89.5% F1, a full 23.2 points higher. The same model architecture trained on SQuAD gets 85.8% F1, only 5.4 points worse than humans. We also show that our unanswerable questions are more challenging than ones created automatically, either via distant supervision (Clark and Gardner, 2017) or a rule-based method (Jia and Liang, 2017). We release SQUADRUN to the public as the successor to SQuAD, and designate it SQuAD 2.0 on the official SQuAD leaderboard.² We are optimistic that this new dataset will encourage the development of reading comprehension systems that know what they don’t know.

2 Desiderata

We first outline our goals for SQUADRUN. Besides the generic goals of large size, diversity, and low noise, we posit two desiderata specific to unanswerable questions:

Relevance. The unanswerable questions should appear relevant to the topic of the context paragraph. Otherwise, simple heuristics (e.g., based on word overlap) could distinguish answerable and unanswerable questions (Yih et al., 2013).

Existence of plausible answers. There should be some span in the context whose type matches the type of answer the question asks for. For example, if the question asks, “*What company was founded in 1992?*”, then some company should be mentioned in the context. Otherwise, type-matching heuristics could distinguish answerable and unanswerable questions (Weissenborn et al., 2017).

3 Existing datasets

Next, we survey existing reading comprehension datasets with these criteria in mind. We use the

² As with previous versions of SQuAD, we release SQUADRUN under the CC BY-SA 4.0 license.

term “negative example” to refer to a context passage paired with an unanswerable question.

3.1 Extractive datasets

In extractive reading comprehension datasets, a system must extract the correct answer to a question from a context document or paragraph. The Zero-shot Relation Extraction dataset (Levy et al., 2017) contains negative examples generated with distant supervision. Levy et al. (2017) found that 65% of these negative examples do not have a plausible answer, making them easy to identify.

Other distant supervision strategies can also create negative examples. TriviaQA (Joshi et al., 2017) retrieves context documents from the web or Wikipedia for each question. Some documents do not contain the correct answer, yielding negative examples; however, these are excluded from the final dataset. Clark and Gardner (2017) generate negative examples for SQuAD by pairing existing questions with other paragraphs from the same article based on TF-IDF overlap; we refer to these as TFIDF examples. In general, distant supervision does not ensure the existence of a plausible answer in the retrieved context, and might also add noise, as the context might contain a paraphrase of the correct answer. Moreover, when retrieving from a small set of possible contexts, as in Clark and Gardner (2017), we find that the retrieved paragraphs are often not very relevant to the question, making these negative examples easy to identify.

The NewsQA data collection process also yields unanswerable questions, because crowdworkers write questions given only a summary of an article, not the full text (Trischler et al., 2017). Only 9.5% of their questions are unanswerable, making this strategy hard to scale. Of this fraction, we found that some are misannotated as unanswerable, and others are out-of-scope (e.g., summarization questions). Trischler et al. (2017) also exclude negative examples from their final dataset.

Jia and Liang (2017) propose a rule-based procedure for editing SQuAD questions to make them unanswerable. Their questions are not very diverse: they only replace entities and numbers with similar words, and replace nouns and adjectives with WordNet antonyms. We refer to these unanswerable questions as RULEBASED questions.

3.2 Answer sentence selection datasets

Sentence selection datasets test whether a system can rank sentences that answer a question higher

Reasoning	Description	Example	Percentage
Negation	Negation word inserted or removed.	Sentence: “Several hospital pharmacies have decided to outsource high risk preparations . . .” Question: “What types of pharmacy functions have never been outsourced?”	9%
Antonym	Antonym used.	S: “the extinction of the dinosaurs. . . allowed the tropical rainforest to spread out across the continent.” Q: “The extinction of what led to the decline of rainforests?”	20%
Entity Swap	Entity, number, or date replaced with other entity, number, or date.	S: “These values are much greater than the 9–88 cm as projected . . . in its Third Assessment Report.” Q: “What was the projection of sea level increases in the fourth assessment report ?”	21%
Mutual Exclusion	Word or phrase is mutually exclusive with something for which an answer is present.	S: “BSkyB. . . waiv[ed] the charge for subscribers whose package included two or more premium channels.” Q: “What service did BSKyB give away for free unconditionally ?”	15%
Impossible Condition	Asks for condition that is not satisfied by anything in the paragraph.	S: “Union forces left Jacksonville and confronted a Confederate Army at the Battle of Olustee. . . Union forces then retreated to Jacksonville and held the city for the remainder of the war.” Q: “After what battle did Union forces leave Jacksonville for good ?”	4%
Other Neutral	Other cases where the paragraph does not imply any answer.	S: “Schuenemann et al. concluded in 2011 that the Black Death. . . was caused by a variant of <i>Y. pestis</i> . . .” Q: “Who discovered <i>Y. pestis</i> ?”	24%
Answerable	Question is answerable (i.e. dataset noise).		7%

Table 1: Types of negative examples in SQUADRUN exhibiting a wide range of phenomena.

than sentences that do not. Wang et al. (2007) constructed the QASENT dataset from questions in the TREC 8-13 QA tracks. Yih et al. (2013) showed that lexical baselines are highly competitive on this dataset. WikiQA (Yang et al., 2015) pairs questions from Bing query logs with sentences from Wikipedia. Like TFIDF examples, these sentences are not guaranteed to have plausible answers or high relevance to the question. The dataset is also limited in scale (3,047 questions, 1,473 answers).

3.3 Multiple choice datasets

Finally, some datasets, like MCTest (Richardson et al., 2013) and RACE (Lai et al., 2017), pose multiple choice questions, which can have a “none of the above” option. In practice, multiple choice options are often unavailable, making these datasets less suited for training user-facing systems. Multiple choice questions also tend to be quite different from extractive ones, with more emphasis on fill-in-the-blank, interpretation, and summarization (Lai et al., 2017).

4 The SQUADRUN dataset

We now describe our new dataset, which we constructed to satisfy both the relevance and plausible answer desiderata from Section 2.

4.1 Dataset creation

We employed crowdworkers on the Daemo crowdsourcing platform (Gaikwad et al., 2015) to write unanswerable questions. Each task consisted of an entire article from the original SQuAD dataset. For each paragraph in the article, workers were asked to pose up to five questions that were impossible to answer based on the paragraph alone, while referencing entities in the paragraph and ensuring that a plausible answer is present. As inspiration, we also showed questions from SQuAD for each paragraph; this further encouraged unanswerable questions to look similar to answerable ones. Workers were asked to spend 7 minutes per paragraph, and were paid \$10.50 per hour. Screenshots of our interface are shown in Appendix A.1.

We removed questions from workers who wrote 25 or fewer questions on that article; this filter helped remove noise from workers who had trouble understanding the task, and therefore quit before completing the whole article. We applied this filter to both our new data and the existing answerable questions in SQuAD. To generate train, development, and test splits, we used the same partition of articles as SQuAD, and combined the existing SQuAD data with our new data for each split. For the SQUADRUN development and test sets, we removed articles for which we did not col-

	SQuAD	SQUADRUN
Train		
Total examples	87,599	130,319
Negative examples	0	43,498
Total articles	442	442
Articles with negatives	0	285
Development		
Total examples	10,570	11,873
Negative examples	0	5,945
Total articles	48	35
Articles with negatives	0	35
Test		
Total examples	9,533	8,862
Negative examples	0	4,332
Total articles	46	28
Articles with negatives	0	28

Table 2: Dataset statistics of SQUADRUN, compared to the original SQuAD dataset.

lect unanswerable questions. This resulted in a roughly one-to-one ratio of answerable to unanswerable questions in these splits, whereas the train data has roughly twice as many answerable questions as unanswerable ones. Table 2 summarizes overall statistics of SQUADRUN.

4.2 Human accuracy

To confirm that our dataset is clean, we hired additional crowdworkers to answer all questions in the SQUADRUN development and test sets. In each task, we showed workers an entire article from the dataset. For each paragraph, we showed all associated questions; unanswerable and answerable questions were shuffled together. For each question, workers were told to either highlight the answer in the paragraph, or mark it as unanswerable. Workers were told to expect every paragraph to have some answerable and some unanswerable questions. They were asked to spend one minute per question, and were paid \$10.50 per hour.

To reduce crowdworker noise, we collected multiple human answers for each question and selected the final answer by majority vote, breaking ties in favor of answering questions and preferring shorter answers to longer ones. On average, we collected 4.8 answers per question. We note that for the original SQuAD, Rajpurkar et al. (2016) evaluated a single human’s performance; therefore, they likely underestimate human accuracy.

4.3 Analysis

We manually inspected 100 randomly chosen negative examples from our development set to understand the challenges these examples present. In Table 1, we define different categories of nega-

tive examples, and give examples and their frequency in SQUADRUN. We observe a wide range of phenomena, extending beyond expected phenomena like negation, antonymy, and entity changes. In particular, SQUADRUN is much more diverse than RULEBASED, which creates unanswerable questions by applying entity, number, and antonym swaps to existing SQuAD questions. We also found that 93% of the sampled negative examples are indeed unanswerable.

5 Experiments

5.1 Models

We evaluated three existing model architectures: the BiDAF-No-Answer (BNA) model proposed by Levy et al. (2017), and two versions of the DocumentQA No-Answer (DocQA) model from Clark and Gardner (2017), namely versions with and without ELMo (Peters et al., 2018). These models all learn to predict the probability that a question is unanswerable, in addition to a distribution over answer choices. At test time, models abstain whenever their predicted probability that a question is unanswerable exceeds some threshold. We tune this threshold separately for each model on the development set. When evaluating on the test set, we use the threshold that maximizes F1 score on the development set. We find this strategy does slightly better than simply taking the argmax prediction, possibly due to the different proportions of negative examples at training and test time.

5.2 Main results

First, we trained and tested all three models on SQUADRUN, as shown in Table 3. Following Rajpurkar et al. (2016), we report average exact match and F1 scores.³ The best model, DocQA + ELMo, achieves only 66.3 F1 on the test set, 23.2 points lower than the human accuracy of 89.5 F1. Note that a baseline that always abstains gets 48.9 test F1; existing models are closer to this baseline than they are to human performance. Therefore, we see significant room for model improvement on this task. We also compare with reported test numbers for analogous model architectures on SQuAD. There is a much larger gap between humans and machines on SQUADRUN compared to SQuAD, which confirms that SQUADRUN is a much harder dataset for existing models.

³ For negative examples, abstaining receives a score of 1, and any other response gets 0, for both exact match and F1.

System	SQuAD test		SQUADRUN dev		SQUADRUN test	
	EM	F1	EM	F1	EM	F1
BNA	68.0	77.3	59.8	62.6	59.2	62.1
DocQA	72.1	81.0	61.9	64.8	59.3	62.3
DocQA + ELMo	78.6	85.8	65.1	67.6	63.4	66.3
Human	82.3	91.2	86.3	89.0	86.9	89.5
Human–Machine Gap	3.7	5.4	21.2	21.4	23.5	23.2

Table 3: Exact Match (EM) and F1 scores on SQUADRUN and SQuAD. The gap between humans and the best tested model is much larger on SQUADRUN, suggesting there is a great deal of room for model improvement.

System	SQuAD + TfIDF		SQuAD + RULEBASED		SQUADRUN dev	
	EM	F1	EM	F1	EM	F1
BNA	72.7	76.6	80.1	84.8	59.8	62.6
DocQA	75.6	79.2	80.8	84.8	61.9	64.8
DocQA + ELMo	79.4	83.0	85.7	89.6	65.1	67.6

Table 4: Exact Match (EM) and F1 scores on the SQUADRUN development set, compared with SQuAD with two types of automatically generated negative examples. SQUADRUN is more challenging for current models.

5.3 Automatically generated negatives

Next, we investigated whether automatic ways of generating negative examples can also yield a challenging dataset. We trained and tested all three model architectures on SQuAD augmented with either TfIDF or RULEBASED examples. To ensure a fair comparison with SQUADRUN, we generated training data by applying TfIDF or RULEBASED only to the 285 articles for which SQUADRUN has unanswerable questions. We tested on the articles and answerable questions in the SQUADRUN development set, adding unanswerable questions in a roughly one-to-one ratio with answerable ones. These results are shown in Table 4. The highest score on SQUADRUN is 15.4 F1 points lower than the highest score on either of the other two datasets, suggesting that automatically generated negative examples are much easier for existing models to detect.

5.4 Plausible answers as distractors

Finally, we measured how often systems were fooled into answering the plausible but incorrect answers provided by crowdworkers for our unanswerable questions. For both computer systems and humans, roughly half of all wrong answers on unanswerable questions exactly matched the plausible answers. This suggests that the plausible answers do indeed serve as effective distractors. Full results are shown in Appendix A.2.

6 Discussion

SQUADRUN forces models to understand whether a paragraph entails that a certain span is the answer to a question. Similarly, recognizing

textual entailment (RTE) requires systems to decide whether a hypothesis is entailed by, contradicted by, or neutral with respect to a premise (Marelli et al., 2014; Bowman et al., 2015). Relation extraction systems must understand when a possible relationship between two entities is not entailed by the text (Zhang et al., 2017).

Jia and Liang (2017) created adversarial examples that fool pre-trained SQuAD models at test time. However, models that train on similar examples are not easily fooled by their method. In contrast, the adversarial examples in SQUADRUN are difficult even for models trained on examples from the same distribution.

In conclusion, we have presented SQUADRUN, a challenging, diverse, and large-scale dataset that forces models to understand when a question cannot be answered given the context. We are optimistic that SQUADRUN will encourage the development of new reading comprehension models that know what they don’t know, and therefore understand language at a deeper level.

Reproducibility. All code, data, experiments are available on the Codalab platform at <https://bit.ly/2rDHBgY>.

Acknowledgments. We would like to thank the anonymous reviewers, Arun Chaganty, Peng Qi, and Sharon Zhou for their constructive feedback. We are grateful to Durim Morina and Michael Bernstein for their help with the Daemon platform. This work was supported by funding from Facebook. R.J. is supported by an NSF Graduate Research Fellowship under Grant No. DGE-114747.

References

- S. Bowman, G. Angeli, C. Potts, and C. D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- C. Clark and M. Gardner. 2017. Simple and effective multi-paragraph reading comprehension. *arXiv preprint arXiv:1710.10723*.
- S. N. Gaikwad, D. Morina, R. Nistala, M. Agarwal, A. Cossette, R. Bhanu, S. Savage, V. Narwal, K. Rajpal, J. Regino, et al. 2015. Daemon: A self-governed crowdsourcing marketplace. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology*. pages 101–102.
- K. M. Hermann, T. Koisk, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, and P. Blunsom. 2015. Teaching machines to read and comprehend. In *Advances in Neural Information Processing Systems (NIPS)*.
- D. Hewlett, A. Lacoste, L. Jones, I. Polosukhin, A. Fandrianto, J. Han, M. Kelcey, and D. Berthelot. 2016. Wikireading: A novel large-scale language understanding task over Wikipedia. In *Association for Computational Linguistics (ACL)*.
- M. Hu, Y. Peng, and X. Qiu. 2017. Reinforced mnemonic reader for machine comprehension. *arXiv*.
- H. Huang, C. Zhu, Y. Shen, and W. Chen. 2018. Fusionnet: Fusing via fully-aware attention with application to machine comprehension. In *International Conference on Learning Representations (ICLR)*.
- R. Jia and P. Liang. 2017. Adversarial examples for evaluating reading comprehension systems. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Association for Computational Linguistics (ACL)*.
- G. Lai, Q. Xie, H. Liu, Y. Yang, and E. Hovy. 2017. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*.
- O. Levy, M. Seo, E. Choi, and L. Zettlemoyer. 2017. Zero-shot relation extraction via reading comprehension. In *Computational Natural Language Learning (CoNLL)*.
- M. Marelli, S. Menini, M. Baroni, L. Bentivogli, R. bernardi, and R. Zamparelli. 2014. A SICK cure for the evaluation of compositional distributional semantic models. In *Language Resources and Evaluation Conference (LREC)*.
- T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, and L. Deng. 2016. MS MARCO: A human generated machine reading comprehension dataset. In *Workshop on Cognitive Computing at NIPS*.
- M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer. 2018. Deep contextualized word representations. In *North American Association for Computational Linguistics (NAACL)*.
- P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- M. Richardson, C. J. Burges, and E. Renshaw. 2013. Mctest: A challenge dataset for the open-domain machine comprehension of text. In *Empirical Methods in Natural Language Processing (EMNLP)*. pages 193–203.
- M. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi. 2016. Bidirectional attention flow for machine comprehension. *arXiv*.
- A. Trischler, T. Wang, X. Yuan, J. Harris, A. Sordoni, P. Bachman, and K. Suleman. 2017. NewsQA: A machine comprehension dataset. In *Workshop on Representation Learning for NLP*.
- M. Wang, N. A. Smith, and T. Mitamura. 2007. What is the jeopardy model? a quasi-synchronous grammar for QA. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- W. Wang, N. Yang, F. Wei, B. Chang, and M. Zhou. 2017. Gated self-matching networks for reading comprehension and question answering. In *Association for Computational Linguistics (ACL)*.
- D. Weissenborn, G. Wiese, and L. Seiffe. 2017. Making neural QA as simple as possible but not simpler. In *Computational Natural Language Learning (CoNLL)*.
- Y. Yang, W. Yih, and C. Meek. 2015. WikiQA: A challenge dataset for open-domain question answering. In *Empirical Methods in Natural Language Processing (EMNLP)*. pages 2013–2018.
- W. Yih, M. Chang, C. Meek, and A. Pastusiak. 2013. Question answering using enhanced lexical semantic models. In *Association for Computational Linguistics (ACL)*.
- Y. Zhang, V. Zhong, D. Chen, G. Angeli, and C. D. Manning. 2017. Position-aware attention and supervised data improve slot filling. In *Empirical Methods in Natural Language Processing (EMNLP)*.